ZYVORAI LABS · USER DOCUMENTATION

Zyvor Fabric

Page-by-Page Product Manual

2026-09-10

zyvor.dev · Apache License 2.0

Page-by-page guides

Each guide follows: Purpose → When to use it → How to get there → What you can do → Related pages.

Every route is also listed in the complete page index.

Auth

Page	What it covers
Sign in	Sign-in screen for Zyvor Fabric. Authenticates against the local admin account or a configured identity provider / PAM user on the host.

Core

Page	What it covers
Create VM	Create VM — a three-step wizard (Basics → Resources → Review) for launching a new virtual machine, including how it's networked and whether it's reachable from outside the host.
Datacenters	Datacenters — the physical inventory tree: datacenters, the clusters inside each one, and the hosts registered to each cluster, with live CPU/memory usage and VM counts per host.
Favorites	Favorites — a personal, starred shortlist of VMs pulled from your full VM list, so the machines you use most are one click away instead of buried in a longer list.
Dashboard	Dashboard — the fabric at a glance: how many VMs exist and in what state, live CPU/memory trends, and whether each backend subsystem is actually reachable.
Machines (removed)	
Profiles	Profiles (shown in the UI as Instance Types) — a library of VM sizing presets (vCPUs, memory, disk, and optionally network bandwidth) you can pick instead of hand-tuning resources every time you create a VM.
Settings	Settings — Core product and console preferences: daemon identity and log level, auto-refresh, default bridge/DNS, default storage pool and disk format, snapshot retention, auth/TLS/session/audit toggles, and notification hooks (email / webhook / VM lifecycle events).
VM Browser	VM Browser — a lightweight, read-only grid of every VM, for quickly scanning or searching without the bulk-action tooling of the full Virtual Machines list.
VM Wizard	VM Wizard — legacy guided VM creation route. The console now uses the three-step Create VM wizard; `/app/vm-wizard` redirects there automatically.
VM Console	VM Console — a real, live console into a running VM, from the browser, without SSH or any other client installed.
Virtual Machines	Virtual Machines — the fleet view of every VM in the fabric, and the starting point for managing any one of them.

Infrastructure

Page	What it covers
Containers	Containers — a read-only, auto-refreshing view of container workloads running on the host (Docker/Podman-style containers, distinct from VMs), showing per-container state, image,

Page	What it covers
	CPU/memory usage, and network I/O.
VM Dataplane (Network Fabric schema v4)	Per-VM edge security and telemetry powered by FluxVM Network Fabric
Distributed Storage	Distributed Storage — the enterprise/clustered layer above a single storage backend: replicated pools spanning multiple hosts, storage policies (tiering, replication, encryption/dedup/compression), in-flight VM disk migrations between pools, and datastore clusters with automatic space/latency-based balancing. For creating and starting a single NFS/LVM/ZFS/Ceph pool, see Storage Pools; for browsing volumes inside pools, see Storage.
Edge Dataplane (cluster)	Cluster-wide console for FluxVM Network Fabric schema v4 (security groups,
Net Security	Net Security — the advanced SDN and security control plane: network policies scoped to security identities, host firewall profiles/zones/VM assignments, exposed services, QoS traffic shaping, DNS zones/policies, WireGuard VPN tunnels and networks, traffic mirroring, NAT rules/pools/gateways, and bandwidth monitoring with alerts.
Network	Network — day-to-day VM networking: which mode a VM uses, its port forwards, and its assigned address. For the advanced SDN stack (policies, firewalls, VPN mesh, QoS, mirroring), see Net Security. For per-VM TC/eBPF edge allowlists, rate limits, and flows, see VM Dataplane.
Resource Pools	Resource Pools — hierarchical CPU/memory allocation pools (nested, with shares, reservations, and limits) that VMs draw from, plus an admission-control test to check whether a workload's requirements would fit before you commit to it.
Storage Pools	Storage Pools — create, start/stop, and monitor the storage backends VM disks live on: local directories, NFS exports, LVM and LVM-thin volume groups, ZFS pools, or Ceph RBD pools. This is where a pool's lifecycle and health live; for the volumes (disks) inside those pools see Storage, and for multi-host replicated/policy-driven storage see Distributed Storage.
Storage	Storage — a consolidated view of every storage pool's capacity alongside a manual volume tracking ledger. To create or manage a pool itself, use Storage Pools; for replicated/policy-driven storage across hosts, see Distributed Storage.
System Health	System Health — a live, read-only dashboard of host resource utilization, refreshing every 2 seconds: CPU, memory, disk I/O, filesystems, network interfaces, and top processes, rolled up into a single health score.
System	System — the physical host's hardware topology (CPU sockets/cores/threads, NUMA nodes, hugepages) plus topology-aware optimization recommendations for individual VMs. This is distinct from System Health, which tracks live utilization rather than hardware layout.
Warm Pools (VM Pools)	Warm Pools — keep N VMs pre-booted and paused from a chosen disk image so you can claim one instantly instead of cold-creating. Each pool shows ready members vs size, vCPUs/memory, and the image path.
Availability Zones	Zones — availability / placement domains for the fabric: named zones (region, status, host count) plus spot instances (max price, priority, eviction policy) that can be requested, evicted, or deleted against those zones.

Monitoring

Page	What it covers
Alerts	Alerts — a live view of currently firing system alerts and the notification rules that generate them, polling for updates automatically.
Analytics	Performance Analytics — fleet-wide resource utilization, trends over time, and per-VM performance insights, with exportable reports.
Audit	Audit Logs — the security and compliance trail of who did what: every tracked action, which user performed it, on which resource, whether it succeeded, and from what IP address.
Capacity	Capacity Planning — resource usage against total capacity per resource (memory, CPU, storage), with week-over-week trend, so you can see what's running out and when.
Debug Tools	Debug Tools — raw, terminal-style output from four classic Linux diagnostic commands (top, iostat, vmstat, netstat) against the host, rendered as monospace panels. This is the closest the dashboard gets to SSHing in and running commands yourself.
Event Stream	Event Stream — a live, scrolling log of VM lifecycle events (create, start, stop, delete, and similar) pushed over an authenticated SSE connection as they happen. There's no history — you only see events that occur while the page is open.
Explain	Explain — plain-language, AI-generated explanations for a chosen system metric: its current value and trend, an assessment of its status, what's contributing to it, and what to do about it. This is the interpretive layer on top of the raw numbers you'd see in Analytics or Debug Tools.
Kernel	Kernel — a snapshot of the host's kernel configuration: version, hostname, architecture, boot command line, loaded kernel modules, and sysctl parameters. This is static configuration, not live activity — for that, see Debug Tools or Event Stream.
Live Metrics	Live Metrics — a real-time view of host performance: CPU, memory, disk I/O, and network throughput, each as a rolling sparkline that updates once per second.
Logs	Logs — a searchable, filterable console view of Zyvor Fabric's audit log: every recorded action and event, level-coded and continuously refreshed.
Notifications	Notifications — configure how and when Zyvor Fabric alerts you: the delivery channels (email, Slack, Teams, webhook), the rules that trigger them, and a history of what was actually sent.
Processes	Processes — a live process monitor for the host: every OS process with its CPU and memory usage, refreshed every 3 seconds, with a per-process detail drill-down.
Optimizer	Optimizer — a right-sizing advisor that analyzes each VM's actual resource usage and recommends CPU/memory/disk adjustments, with a one-click apply per VM.
Service Map	Service Map — shows the services discovered across your VMs, which ones depend on each other (protocol and port), and each service's current health.
Timeline	Timeline — a single reverse-chronological activity feed that merges audit-log actions and system alerts, so you can see what happened and in what order without switching between Logs and Notifications.

More Images Migrations Managers

Page	What it covers
Backup Scheduler	Backup Scheduler — create and manage recurring, automated backup jobs that snapshot one or more VMs' disks to a directory on a schedule, with retention and format controls.

Page	What it covers
Batch Import	Batch Import — bulk-create VMs from a YAML or JSON list, with a preview step and a per-VM status readout as each one is submitted.
Batch Migration	Batch Migration Builder — a form-based editor for assembling a multi-VM migration job spec (source disk, target format, sizing) and exporting it as JSON. It builds the spec only; it does not run the migration itself.
Disk Converter	Disk Format Converter — convert a single disk image between QCOW2, VMDK, VHD, VHDX, and RAW, tracking the conversion job's progress to completion.
Disk Images	Disk Images — a read-only inventory of the VM disk images present on the host: name, format, size, and path for each one.
Download Disk	Download Disk — browse the disk images available on the Fabric host and download any of them straight to your machine.
Image Builder	Image Builder — build custom VM disk images from scratch using mkosi, by picking a Linux distribution and a package list, and track builds from queued through to a finished image.
ISO Images	ISO Images — a read-only inventory of installer and driver ISO files sitting in the host's configured images directory, showing which VMs currently have each one attached.
Job Monitor	Job Monitor — a live view of background jobs (disk conversions, migrations, and other pipeline work), with per-job progress, pipeline stage, and streaming logs.
Manifest Builder	Manifest Builder — a client-side form for assembling a VM configuration manifest and exporting it as YAML, with a live preview as you type. It doesn't create a VM or call the API; it's a scratchpad for drafting config to copy elsewhere.
History	Migration History — a read-only log of completed and failed migration jobs, with status, timing, and where the output landed.
Readiness	Migration Readiness — pre-flight checks that verify the environment is in a good state before you start a migration, with a pass/fail summary and per-check detail.
Report	Migration Report — a shareable summary of all migration jobs (totals by status, average duration) plus the full per-migration detail table, with copy and print actions.
Migration Templates	Migration Templates — reusable migration configuration presets (disk format, vCPUs, memory, network, compression) that you can copy as JSON instead of re-entering the same settings for every migration.
Network Topology	Network Topology — a live map of which VMs are attached to which virtual networks or host bridges, alongside the host's own bridges and physical NICs, auto-refreshing every 15 seconds.
Pipeline	Pipeline Monitor — a live, auto-refreshing view of in-progress migration/conversion jobs, showing each job's percent complete and which of five stages it's currently in.
Snapshot Mgr	Snapshot Manager — create, revert to, and delete disk-state snapshots for a selected VM.
Storage Mgr	Storage Manager — browse storage pools with their capacity and usage, and drill into a pool to see its volumes.
Upload Disk	Upload Disk Image — drag-and-drop (or browse) upload of a VM disk image file to the server, with live progress and an in-session upload history.

Operations

Page	What it covers
Autoscale	Autoscale — define per-VM policies that automatically grow or shrink a VM's vCPUs and memory within set bounds based on load, and review the history of scaling actions that were triggered.
Backups	Backups & Restore — create full or incremental VM backups, track running backup/restore jobs, and restore a VM from a completed backup, either in place or as a new VM.
Bulk Operations	Bulk Operations — select any number of VMs and start, stop, restart, or snapshot them together, with a per-VM progress log for the batch.
Content Library	Content Library — a catalog of reusable provisioning building blocks: libraries of templates/ISOs/OVFs/scripts, guest customization specs (per-OS hostname/domain/DNS settings), and host compliance profiles.
DRS	Distributed Resource Scheduler (DRS) — balances VM placement across the hosts in a cluster, surfaces migration recommendations, enforces affinity/anti-affinity rules, and can test where a new VM would land before you create it.
Fault Tolerance	Fault Tolerance (FT) — protect individual VMs with a live secondary replica on another host, so a host failure fails the VM over instead of taking it down, and monitor replication health.
Lifecycle	Lifecycle Manager — define patch/upgrade baselines, scan hosts for compliance against them, remediate non-compliant hosts, and track rolling updates across a host fleet.
Migration Wizard	Migration Wizard — a three-step wizard (Source → Configure → Review) for converting an existing disk image (local file or remote host) into a new Zyvor Fabric VM.
Migrations	VM Migrations — move a VM from its current host to a different target host, and track the migration from start to finish. The list auto-refreshes every 5 seconds so in-flight migrations update live.
Quotas	Resource Quotas — cap CPU, memory, disk, and VM-count usage, applied either globally or to VMs matching specific tags, so a team or workload can't consume unlimited host resources.
Replication	Replication — register remote replication sites, configure per-VM replication to them with a target RPO (recovery point objective), and monitor sync health and RPO compliance across your fleet.
Schedules	VM Schedules — automate a recurring lifecycle action (start, stop, restart, or snapshot) for a single VM, on a one-time, daily, or weekly schedule.
Site Recovery	Site Recovery — define disaster recovery plans that group VMs by source and target site, then execute those plans as a test failover, a planned migration, or a full disaster recovery, and track how each execution unfolds.
Snapshots	VM Snapshots — create point-in-time snapshots of a specific VM's disk (or disk + memory), and revert or delete them. Unlike most Operations pages, this one is scoped to one VM at a time, entered by name.
Templates	VM Templates — reusable VM configurations (CPU/memory/disk, tags) that you save from an existing VM and use to stamp out new VMs quickly, instead of configuring resources from scratch each time.

Security

Page	What it covers
Access Control	Access Control — manage the user accounts that can sign in to Zyvor Fabric: create accounts, assign a role (admin, operator, or viewer), and enable/disable or delete them.

Page	What it covers
Certificates	Certificates & Security — a PKI console for Zyvor Fabric: certificate authorities, issued certificates, the CSR approval queue, host TPM/boot attestation, and VM security baselines, rolled up into one health dashboard.
Compliance	Compliance Dashboard — a security and configuration compliance scorecard: an overall score, pass/warning/fail counts by category, and remediation guidance for anything that isn't passing.
Encryption	Encryption — manage VM disk and vMotion encryption: register key management providers (KMIP, HashiCorp Vault Transit, or local software keys), define reusable encryption policies, and see which VMs are encrypted under which policy.
Plugins	Plugin Manager — enable, disable, and review the server extensions installed on Zyvor Fabric (storage, network, security, monitoring, and backup plugin types).
Security Dashboard	Security — a real-time security posture and threat-monitoring view: an overall risk score, active security alerts, recent failed login attempts, and listening network ports on the host. Data auto-refreshes every 5 seconds.

Tools

Page	What it covers
Cost Estimator	Storage Cost Estimator — a what-if calculator that projects cloud storage cost (AWS S3, Azure Blob, GCS) for your VM fleet and compares it against an on-premises baseline. It's a planning tool: figures come from the inputs you set, not from your actual billing or usage.
Notification Center	Notification Center — a live, session-only tray of VM events and system alerts, polled from the server every 10 seconds. It's separate from Monitoring → Notifications, which manages persistent delivery channels, rules, and history; this page just surfaces what's firing right now and forgets everything when you reload.
API Playground	API Playground — send authenticated, ad-hoc HTTP requests to the Zyvor Fabric API from the browser: pick a preset endpoint or type any path, choose method, optional JSON body, inspect status/headers/body, and keep a short in-session history.
VM Compare	VM Comparison — a side-by-side diff of two VMs' configurations, run on demand against the live VM list.
VM Health Check	VM Health Check — runs a set of health verification checks against a single VM, on demand, and reports pass/warning/fail per check plus an overall status.
Webhooks	Webhook Configuration — manage outbound webhooks that notify an external endpoint (generic HTTP, Slack, or Discord) when specific VM and backup events occur.

---

86 guides. Regenerate: `node scripts/user-docs/generate-guide-index.mjs`.

Sign in

Purpose

Sign-in screen for Zyvor Fabric. Authenticates against the local admin account or a configured identity provider / PAM user on the host.

When to use it

- Whenever you're not signed in — protected `/app` routes redirect here if the session is missing or expired
- To sign in as the local admin (`admin` + generated password)
- To sign in with your own system account when PAM/OIDC is configured
- After an expired JWT session when the console sends you back here

How to get there

- Route: `/sign-in` (legacy `/login` redirects here)
- From marketing pages: **Sign in** in the top nav
- After signing in you land on the console at `/app`

Operate from the console (UX)

1. Enter a **username** and click **Continue**.
2. Enter your **password** and sign in — a failed attempt shows an inline error.
3. On success you are taken to `/app` (dashboard).
4. Two common ways to sign in: **local admin** — username `admin`, password from `./zyvor-fabricd-ctl password` or `/var/lib/zyvor-fabricd/.admin_password` — or your own **system user** account when PAM/OIDC is configured.

Empty / fail: Inline error on bad credentials, or the page never loads — confirm `zyvor-fabricd` is reachable on `:9095` and that you have the current admin password (Admin basics).

Success: You land on `/app` with the console nav visible.

Related pages

- Dashboard
- Access Control
- Getting Started
- Admin basics
- Page index

Create VM

Purpose

Create VM — a three-step wizard (Basics → Resources → Review) for launching a new virtual machine, including how it's networked and whether it's reachable from outside the host.

When to use it

- To provision a new VM from a catalog image or a custom disk path
- To decide up front how the VM will be reachable — private-and-forwarded, or a real address on your network
- Start from the product home / dashboard if you are unsure where to begin

How to get there

- Route / id: `/app/create`
- Nav: **Core → Create VM** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Basics — name the VM and pick a disk image, either from the built-in catalog or a path on the host.

2. Resources — set vCPUs, memory, and root disk size. Under **Advanced Options** you choose networking:

- **NAT (default)** — the VM gets a private, outbound-only address, same as most VMs today. To make a service on the VM reachable from your laptop or the wider network, use **Expose ports**: add a host port → guest port mapping (there's a one-click **Expose SSH (22)** preset for the common case of just needing to SSH in). Forwarded ports are reachable from any client that can reach the host — not just from the host itself.
- **Bridged** — the VM gets its own real address on the local network via a dedicated network namespace, instead of sharing the host's NAT. Two addressing options:
  - **DHCP** (default under Bridged) — the VM's guest agent leases an address automatically.
  - **Assign the IP statically via cloud-init** — check this box to bake a fixed address into the VM at boot instead of waiting on DHCP, useful when the guest image doesn't run a DHCP client or you need a predictable address before first boot.

3. Review — confirms name, image, resources, and the networking mode you chose, then creates the VM.

After creation — port forwards aren't locked in at creation time: open the VM's **Network** tab from its detail page to add or remove forwards later (the VM restarts automatically if it's running when you do).

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- Virtual Machines
- Network
- Getting Started
- Page index

Datacenters

Purpose

Datacenters — the physical inventory tree: datacenters, the clusters inside each one, and the hosts registered to each cluster, with live CPU/memory usage and VM counts per host.

When to use it

- To see how your infrastructure is organized — datacenter → cluster → host — before deciding where new capacity should go
- To register a new host into a cluster, or stand up a new datacenter or cluster
- To check a host's CPU/memory load, VM count, or connection status
- To put a host into maintenance mode before doing physical work on it

How to get there

- Route / id: `/app/datacenters`
- Nav: **Core → Datacenters** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Read the top-of-page summary cards: total datacenters, clusters, hosts, and VMs across the whole fabric.
2. Click a datacenter to expand it — you'll see a rollup (cluster count, host count, VM count, and, once summary data loads, total CPUs and memory) plus its list of clusters.
3. **Create a datacenter** with the button in the header — a modal asks for a name and optional description.
4. Inside an expanded datacenter, click **+ Cluster** to create a cluster there (name + description), or the trash icon to delete the datacenter — both are confirmed first.
5. Expand a cluster to see its HA and DRS status and a table of its hosts: hostname, address, CPUs, memory, live CPU/memory usage bars, VM count, and status (Connected, Disconnected, Maintenance).
6. Click **+ Host** on a cluster to register a host (hostname, IP address, CPUs, memory in MB), or use the trash icon to remove one — removal is confirmed first.
7. Toggle a host in or out of maintenance mode with the wrench icon next to it.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Dashboard
- Virtual Machines
- Create VM
- Getting Started
- Page index

Favorites

Purpose

Favorites — a personal, starred shortlist of VMs pulled from your full VM list, so the machines you use most are one click away instead of buried in a longer list.

Favorites are **browser-local** (local storage). They do not sync across browsers, devices, or user accounts on the host.

When to use it

- To jump straight to the handful of VMs you check on or console into regularly
- To pin or unpin VMs as your day-to-day working set changes
- To search across every VM (not just the pinned ones) from a single search box
- When the full Virtual Machines list is long and you only need a few daily targets
- After create/start, pin the new VM so it stays visible without hunting by name

How to get there

- Route / id: `/app/favorites`
- Nav: **Core → Favorites** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Open Favorites and wait for the VM list to load from the server (same fleet as `/app/vms`).
2. Click the **star** next to any VM to pin or unpin it. Pinned VMs move into **Pinned VMs**; everything else stays under **All VMs**.
3. Use the **search** box to filter both sections by name or state.
4. Each row shows vCPU count, memory, and a state badge. Click the **name** for the VM detail page, or **Console** for the console tab.
5. **Refresh** reloads the underlying VM list from the server (stars themselves stay in local storage).
6. Clearing site data / using another browser resets the shortlist — re-star the VMs you need.

Typical flow: pin the VMs you will touch in a change window → use search to confirm state → open Console or detail from the pinned section → unpin when the work is done so the shortlist stays small.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Dashboard
- Virtual Machines
- VM Browser
- VM Console
- Create VM

Dashboard

Purpose

Dashboard — the fabric at a glance: how many VMs exist and in what state, live CPU/memory trends, and whether each backend subsystem is actually reachable.

This is the post-sign-in landing page (`/app`). Treat capability chips as the first health gate before change windows.

When to use it

- As your console landing page — it loads at `/app`
- To check overall health before digging into a specific VM or subsystem
- To confirm FluxVM / storage / network / dataplane capability chips before a change window
- Start here if you're new to the product; a first-time install lands somewhere useful, not an empty table
- To verify **VM dataplane** reports Network Fabric **schema=4** when eBPF edge is expected

How to get there

- Route: `/app`
- Nav: **Core → Dashboard** (sidebar or command palette), or the wordmark in the top bar
- After sign-in at `/sign-in` you land here
- Open `http://127.0.0.1:<port>/app` or `https://<host>/app` depending on how you run the console

Operate from the console (UX)

1. Check subsystem status chips (VM driver, storage, network security, **VM dataplane**, authentication, events) — each shows Live, Unreachable, or Off.
2. For **VM dataplane**, a healthy FluxVM Network Fabric edge reads like `mode=ebpf · attached · schema=4`. If schema is missing or not 4, do not rely on Guard/deny policy until VM Dataplane shows Attached + schema 4.
3. Read the stat cards: total VMs, running, stopped, and total allocated memory/vCPUs.
4. Watch live CPU and memory usage charts when VMs are running.
5. Scan the VM table, or open Virtual Machines for full fleet actions.
6. **On a fresh install with no VMs yet**, use Getting Started links to create a VM, templates, API playground, or access control.
7. For Maglev Services / Service Fabric **v6** cluster console, open Edge Dataplane (`/app/edge-dataplane`) — that is separate from the per-VM Network Fabric schema v4 chip.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Virtual Machines
- VM Dataplane

- Edge Dataplane
- Create VM
- Favorites
- System Health
- Getting Started
- Page index

Machines (removed)

The `/app/machines` page and `/api/machines` surface have been **removed**.

Use **Virtual Machines** (`/app/vms`) and the FluxVM-backed VM APIs instead (`GET/POST /api/vms` , lifecycle under `/api/vms/{name}/…`).

Historically this page exposed systemd-machined / machinectl operations; that backend is gone from Fabric.

Profiles

Purpose

Profiles (shown in the UI as **Instance Types**) — a library of VM sizing presets (vCPUs, memory, disk, and optionally network bandwidth) you can pick instead of hand-tuning resources every time you create a VM.

Built-in profiles ship with the product; custom profiles you create are editable/deletable. Built-ins cannot be removed.

When to use it

- To review available instance-type presets and their specs, grouped by category (general, compute, memory, storage, GPU)
- To create a reusable custom profile for a sizing you use often
- To remove a custom profile you no longer need — built-in profiles can't be deleted
- Before opening Create VM, so you know which preset matches the workload
- To standardize team sizing language ("small-dev", "mem-heavy") without retyping numbers

How to get there

- Route / id: `/app/profiles`
- Nav: **Core → Profiles** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Browse profile **cards** — each shows category badge, description, CPUs, memory, disk, and network bandwidth (if set). Built-in cards are labeled **Built-in** and have no delete control.
2. Filter mentally by category (general / compute / memory / storage / GPU) when choosing a preset for a new VM.
3. **Create Profile** — name, category, CPU count, memory (MB), and disk (GB). Submit adds the card to the grid immediately.
4. Delete a **custom** profile with the trash icon on its card. Built-ins stay; the UI simply omits delete for them.
5. After creating a custom profile, use it from the create/wizard flows that offer instance-type selection — this page manages the library, it does not launch VMs by itself.

Typical flow: decide category → create or pick a profile → create the VM from Create/Wizard using that instance type → if quotas apply, confirm the preset still fits under Quotas.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Dashboard
- Virtual Machines
- Create VM
- VM Wizard

- Templates
- Quotas
- Getting Started
- Page index

Settings

Purpose

Settings — Core product and console preferences: daemon identity and log level, auto-refresh, default bridge/DNS, default storage pool and disk format, snapshot retention, auth/TLS/session/audit toggles, and notification hooks (email / webhook / VM lifecycle events).

When to use it

- To change how often the console auto-refreshes live data
- To set the default bridge, DNS servers, or IPv6 preference for new networking
- To pick the default storage pool / disk format / compression and snapshot retention days
- To toggle auth, TLS expectation, session timeout, or audit logging flags exposed in the UI
- To wire a webhook URL and choose which VM events (start / stop / error) should notify

How to get there

- Route / id: `/app/settings`
- Nav: top-bar **Settings** icon, or command palette → “Settings”
- Legacy `/settings` redirects here

Operate from the console (UX)

1. Wait for settings to load from `/api/settings`. A load banner appears if the call fails — use **Retry**.
2. **General** — edit daemon name, log level (`debug` / `info` / `warn` / `error`), enable auto-refresh, and set the refresh interval in seconds.
3. **Network** — default bridge name, comma-separated DNS servers, Enable IPv6 checkbox.
4. **Storage** — default pool (dropdown populated from storage pools when available), default format (e.g. qcow2), compression toggle, snapshot retention days.
5. **Security** — enable auth, enable TLS, session timeout seconds, audit logging.
6. **Notifications** — email notifications toggle, webhook URL, and notify-on-start / stop / error checkboxes.
7. **Save Changes** persists via PUT. **Reset** restores the in-form defaults (save again to persist).
8. **Empty / fail:** Error banner on load, or toast on save failure — confirm you are signed in as admin and `zyvor-fabricd` is healthy (`/readyz`).
9. **Success:** “Settings saved successfully” toast; subsequent page loads show the values you set.

Related pages

- Dashboard
- Storage Pools
- Notifications
- Webhooks
- Getting Started
- Page index

VM Browser

Purpose

VM Browser — a lightweight, read-only grid of every VM, for quickly scanning or searching without the bulk-action tooling of the full Virtual Machines list.

Use this when you need to **find** a VM; use Virtual Machines when you need to **act** on one or many (start/stop, tags, bulk tools).

When to use it

- To browse or search VMs by name, state, or image in a compact card layout
- To jump straight to a single VM's detail page without selection, tag filters, or bulk actions
- When you only need a visual inventory (image, vCPUs, memory, IP) and not lifecycle controls
- Prefer this page when the job matches the purpose above
- Start from the Dashboard (`/app`) if you are unsure where to begin

How to get there

- Route / id: `/app/vm-browser`
- Nav: **Core → VM Browser** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Open the page and note the header counts: total VMs and how many are running.
2. **Search** by name, state, or image — the card grid filters as you type.
3. Each card shows name, state badge, image, vCPU count, memory, and IP (if assigned).
4. Click a card to open that VM's detail page (`/app/vms/:name`).
5. **Refresh** reloads the list from the server.
6. There are no start/stop/snapshot actions here by design — switch to Virtual Machines or Bulk Operations for those.

Typical flow: search by image or state → open the card → continue on detail (console, network, dataplane). For starring a daily set, use Favorites instead of re-searching here.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Dashboard
- Virtual Machines
- Favorites
- Create VM
- VM Console
- Getting Started

VM Wizard

Purpose

VM Wizard — legacy guided VM creation route. The console now uses the three-step **Create VM** wizard; `/app/vm-wizard` redirects there automatically.

When to use it

- You followed an old bookmark or doc that pointed at `/vm-wizard` or `/app/vm-wizard`
- You want the current Basics → Resources → Review create flow (use Create VM)
- Prefer Create VM from Core nav for all new provisioning

How to get there

- Route / id: `/app/vm-wizard` → **redirects to** `/app/create`
- Legacy `/vm-wizard` also redirects into the `/app` create flow
- Nav: use **Core → Create VM** (`/app/create`) — there is no separate Wizard nav item

Operate from the console (UX)

1. Opening `/app/vm-wizard` immediately navigates to `/app/create` — you should not stay on a distinct Wizard page.
2. Complete the Create VM wizard: **Basics** (name + image) → **Resources** (vCPUs, memory, disk, NAT/bridged networking) → **Review** → create.
3. After create, continue on `/app/vms/:name` for console, network, dataplane, and snapshots.
4. **Empty / fail:** If create fails, check FluxVM on the Dashboard capability chips and image availability (see Admin basics).
5. **Success:** Redirect lands on Create VM; after submit, the new VM appears under Virtual Machines.

Related pages

- Create VM — the live wizard
- Virtual Machines
- Profiles
- Getting Started
- Page index

VM Console

Purpose

VM Console — a real, live console into a running VM, from the browser, without SSH or any other client installed.

Two transports share one page: a text **Terminal** (PTY / xterm.js) and a graphical **VNC** session (noVNC). Both use your existing Fabric session — no separate console password or exposed host port.

When to use it

- To reach a VM that isn't network-reachable yet (no IP assigned, no SSH configured)
- To watch or interact with boot output, a serial console, or a full graphical desktop
- To debug a VM that's otherwise unresponsive over the network
- To confirm a guest came up after create/start without leaving the Fabric UI
- During installers or desktop environments where a text shell is not enough (use **VNC**)

How to get there

- Route / id: `/app/vms/:name/console`
- From a VM's detail page, click **Console** in the header or open the Console tab
- Nav: reached via **Core → Virtual Machines**, not linked directly from the top nav
- Shortcuts: Favorites and VM Browser also offer Console / detail jumps

Operate from the console (UX)

The page has two tabs:

1. **Terminal** — interactive shell (xterm.js) streamed over the VM's PTY. Type as you would in any terminal; output renders live.
2. **VNC** — graphical framebuffer (noVNC) for installers, desktops, or anything that draws to the screen.

Operator checks:

3. Confirm the VM is **running** on its detail page before expecting input; wait a few seconds after start if the session connects then stalls (QEMU/FluxVM still coming up).
4. Prefer Terminal for recovery shells and cloud-init logs; switch to VNC when you need a GUI or boot menu.
5. If the frame stays black or disconnects: VM stopped, FluxVM unreachable, or console/VNC backend not ready — check Dashboard VM-driver health and retry.
6. Closing the browser tab ends your view of the session; it does not stop the guest.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Virtual Machines
- Favorites
- VM Browser

- Create VM
- Dashboard
- Getting Started
- Page index

Virtual Machines

Purpose

Virtual Machines — the fleet view of every VM in the fabric, and the starting point for managing any one of them.

When to use it

- To see everything running (and everything stopped) at a glance
- To find a specific VM quickly once you have more than a handful
- To act on several VMs at once (start, stop, delete, archive)
- Start from the product home / dashboard if you are unsure where to begin

How to get there

- Route / id: `/app/vms`
- Nav: **Core → Virtual Machines** (sidebar, command palette, or desktop nav)
- A single VM's detail page is `/app/vms/:name`, reached by clicking its name here

Operate from the console (UX)

On the list

1. Switch between grid and table view with the toggle in the top right.
2. Search by name, image, or tag — the search box also shows "N of M VMs" so you always know if you're looking at a filtered subset.
3. Filter by tag using the tag chips; combine with search.
4. Select one or more VMs with the checkboxes, then use the bulk actions bar to start, stop, delete, or archive them together.
5. Copy a VM's name straight from the list — hover the name and click the copy icon that appears next to it.
6. Click a VM's name to open its detail page.

On a VM's detail page (`/app/vms/:name`)

1. Start, stop, pause, resume, restart, or clone the VM from the header actions.
2. Open a real console — either the browser terminal (xterm.js) or a graphical VNC session — from the Console tab.
3. Manage disks, network (including port forwards), **Dataplane** (FluxVM eBPF edge policy / stats / flows — see VM Dataplane), snapshots, hotplug devices, and cloud-init from their respective tabs.
4. From the **Network** tab, use **Open Dataplane** as a shortcut into the edge policy editor.
5. **Deleting a VM is undoable for a few seconds.** Confirming delete doesn't remove the VM immediately — a bar appears at the bottom of the screen with an **Undo** button and a countdown; the VM is only actually deleted once that countdown finishes unclicked. If you change your mind, click Undo before it runs out.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- Create VM
- VM Console

Containers

Purpose

Containers — a read-only, auto-refreshing view of container workloads running on the host (Docker/Podman-style containers, distinct from VMs), showing per-container state, image, CPU/memory usage, and network I/O.

Monitoring-only — no start/stop/restart/delete actions here. Auto-refresh every 3 seconds.

When to use it

- To check whether container workloads on this host are running, restarting, or exited
- To spot a container consuming excessive CPU or memory before it starves VMs sharing the same host
- To confirm a container's network throughput (RX/TX)
- Prefer this page when the job matches the purpose above
- When Processes shows container runtimes and you want a container-centric view

How to get there

- Route / id: `/app/containers`
- Nav: **Infrastructure → Containers** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Summary cards: total containers, running count, total CPU%, total memory across detected containers.
2. Container grid — name/ID, state badge (running / exited / paused / restarting), image, live CPU/memory bars, RX/TX when available.
3. Auto-refresh every 3 seconds; header refresh forces an immediate reload.
4. Correlate hot containers with host pressure on Live Metrics / System Health.

Typical flow: scan for restarting/exited → note image and resource bars → remediate with your container runtime outside Fabric → confirm the card returns to running. VM workloads stay on Virtual Machines.

Operator tip: high container CPU/memory can starve VMs on the same host — correlate with Live Metrics.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Virtual Machines
- Processes
- Live Metrics
- System Health
- System
- Getting Started
- Page index

VM Dataplane (Network Fabric schema v4)

Purpose

Per-VM **edge** security and telemetry powered by FluxVM Network Fabric **schema v4** (TC/eBPF on the host-visible TAP/netns interface). Use this to:

- Allowlist destinations and L4 ports (`tcp/PORT` , `udp/PORT` , `icmp/0`)
- Deny CIDRs that override allows
- Attach **security groups** / labels and inspect **effective** merged policy
- Cap egress Mbps/PPS and sample flows
- Inspect allow/drop counters — without touching Fabric's host SDN

This is **not** the same as Net Security network policies (label → host nftables). For cluster-wide groups/CNP/health UI, see Edge Dataplane.

When to use it

- Lock down what a sandbox / CI / AI agent VM can reach
- Apply a live deny or rate limit without restarting the guest
- Prove what left the box (stats + flows)
- Confirm eBPF is attached (`schema_version=4`) before relying on `required=true`
- Debug group membership via the **Effective** tab

How to get there

- Route: `/app/vms/:name` → tab **Dataplane**
- Shortcut: VM → **Network** → **Open Dataplane**
- Dashboard: capability card **VM dataplane** (`/app`)
- Cluster console: Edge Dataplane (`/app/edge-dataplane`)

Operate from the console (UX)

Prerequisites

1. FluxVM with `[sandbox.dataplane] mode = "ebpf"` (see operator guide).
2. VM with **bridged / network tap** (`network_tap: true`) so a host `vh…` exists for TC attach.
3. Write permission to change policy (viewers can inspect).

Status

Confirm **Attached = yes**, **Mode = ebpf**, **Schema version = 4**, policy synced, identity, pin dir, and the **Active policy snapshot** (CIDRs, ports, deny, groups, ICMP, Mbps/PPS).

Policy

Cilium-style **packet-flow control** (VM edge only — no Cilium-private maps):

Control	Effect
Open	Default allow, enforcement off
Audit	Evaluate policy, do not drop (`audit_mode`)

Control	Effect
Guard	Default-deny enforcement + flow sampling
Invert	Swap allow/deny CIDRs and flip default allow
Block / **Allow**	Add host or CIDR to deny/allow lists

Also: **Explain** dest:port, **Dry-run Guard**, and policy **templates** — see dataplane-observe-pack.md.

Flows tab **Block** adds that destination to `deny_cidrs`. Cluster-wide Hubble-lite view: Edge Dataplane → Packet flow.

1. Presets (**Allow all** / **Deny all** / **Web egress**) or edit tags.
2. Ports must look like `tcp/443` or `udp/53` (UI validates).
3. Optional **Deny CIDRs**, **Groups**, **Labels**, **FQDNs**, **Entities**, **Allow ICMP**, **Audit mode**.
4. Optional **Max egress Mbps** / **PPS** and **Sample rate** (≥1 for flows).
5. **Save policy** — live map update (brief over-deny possible; never an allow-all gap).

Effective

JSON snapshot of **declared** policy, **membership** (matched groups / identities), and **effective** merged policy (union of CIDRs/ports; tightest rate limits; fail-closed default).

Stats / Flows

Allow vs drop counters; LRU flow table with identity, family, 5-tuple, verdict.

API & CLI (quick)

Action	Surface
Status / policy / stats / flows / effective	`/api/vms/{name}/dataplane/…`
Groups / CNP / health / observe / ipcache / refresh-dns	`/api/dataplane/…`
CLI	`zyvorctl dataplane policy guard|audit|open|invert|block|allow`

Tutorials: Tutorial 09 · edge-dataplane series.

Related pages

- Edge Dataplane — cluster groups/CNP/health/**Packet flow** console
- Network — NAT / bridge / port forwards
- Net Security — host SDN (orthogonal)
- Virtual Machines
- Operator guide: VM edge dataplane
- Page index

Distributed Storage

Purpose

Distributed Storage — the enterprise/clustered layer above a single storage backend: replicated pools spanning multiple hosts, storage policies (tiering, replication, encryption/dedup/compression), in-flight VM disk migrations between pools, and datastore clusters with automatic space/latency-based balancing. For creating and starting a single NFS/LVM/ZFS/Ceph pool, see Storage Pools; for browsing volumes inside pools, see Storage.

When to use it

- To provision a storage pool replicated across multiple hosts with a target replication factor
- To define a storage policy (e.g. a "performance" tier with RF=3 and encryption on) that pools or workloads should conform to
- To watch a VM disk migration between pools and confirm it completed
- To group pools into a datastore cluster that rebalances automatically once a space or IO-latency threshold is crossed

How to get there

- Route / id: `/distributed-storage`
- Nav: **Infrastructure → Distributed Storage** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Four tabs — **Pools**, **Policies**, **Migrations**, **Datastore Clusters** — plus summary cards for total pools, aggregate capacity, policy count, and active migrations.

1. **Pools** — distributed storage pools with a status badge, host count, and replication factor, and a used/available capacity bar. **Create Pool** sets a name, type, and replication factor. Delete a pool with a confirmation dialog.
2. **Policies** — a table of storage policies showing tier, replication factor, failure tolerance, and whether encryption, deduplication, and compression are on. **Create Policy** sets name, description, replication factor, stripe width, failures-to-tolerate, and tier (performance / standard / archive). Delete with confirmation.
3. **Migrations** — a read-only table of VM storage migrations: source pool, target pool, progress %, bytes transferred, and status (pending / in progress / completed / failed).
4. **Datastore Clusters** — a table of clusters showing datastore count, whether storage DRS (SDRS) is enabled, space threshold %, total capacity, and VM count. **Create Cluster** sets a name, space threshold %, and IO-latency threshold (ms) that trigger automatic rebalancing.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Network
- Net Security
- Edge Dataplane
- Getting Started

Edge Dataplane (cluster)

Purpose

Cluster-wide console for FluxVM Network Fabric **schema v4** (security groups, CNP, identities, observe, Hubble-style packet flow, health, ipcache, FQDN refresh) and **Service Fabric v6 (BPF schema 4)** Maglev VIP services (affinity, drain/health, advertisements, conntrack GC, HA delta replication, EDT pacing, FluxScope flows, host-routing).

This is the **VM edge / service** control plane proxied by Fabric (`/api/dataplane/*`). It does **not** replace Net Security (host SDN / `/api/network-policies`).

Per-VM Status / Policy / Effective / Stats / Flows stay on the VM detail Dataplane tab. Service Fabric detail: ebpf-service-fabric.md.

When to use it

- Create or delete shared security groups used by many VMs
- Apply or remove CNP-shaped documents (compiled onto groups)
- Upsert Maglev VIP services (east-west / north-south, NAT/DSR, drain/health, optional `max_egress_mbps` / `flow_sample_rate` / `host_routing`)
- Check dataplane health after upgrades (`ok` , BPF object, bpffs)
- Inspect Service Fabric host schema, backend health, VIP advertisements, flows
- Inspect reserved + group identities and observe endpoints
- Re-resolve FQDN allowlists after DNS changes

How to get there

- Route: `/app/edge-dataplane`
- Nav: **Infrastructure → Edge Dataplane** (beside Net Security)
- Command palette: search “Edge Dataplane”

Sign in at `/sign-in` with the two-step flow: enter username → **Continue** → password.

Operate from the console (UX)

Tab	Actions
Health	Mode, BPF/bpffs presence, group/CNP/ipcache counts, notes
Services	Maglev upsert/delete; schema badge; health reconcile; ads JSON; conntrack GC; HA deltas
Groups	Quick create (name + label) or delete rows
CNP	Paste JSON → Apply; delete listed documents
Identities	Reserved entities + group identities
Observe	Read-only JSON snapshot
Packet flow	Hubble-lite hops with Colorful / Normal theme
Ipcache	Guest IP → identity table

Tab	Actions
Header **Refresh DNS**	`POST /api/dataplane/refresh-dns` (admin); best-effort — succeeds even when FluxVM has no FQDN entries to refresh

Write actions require admin/write role. Soft banner appears when the `vm_dataplane` capability is off or unreachable.

Related pages

- VM Dataplane — per-VM tab
- Service Fabric v6 (BPF schema 4) — ownership, API, leases, HA deltas, identity/L7 policy
- Net Security — host SDN (orthogonal)
- Tutorials: edge-dataplane/
- Operator: fluxvm-dataplane.md
- Page index

Net Security

Purpose

Net Security — the advanced SDN and security control plane: network policies scoped to security identities, host firewall profiles/zones/VM assignments, exposed services, QoS traffic shaping, DNS zones/policies, WireGuard VPN tunnels and networks, traffic mirroring, NAT rules/pools/gateways, and bandwidth monitoring with alerts.

For everyday per-VM networking mode and port forwards, see Network. For **per-VM TC/eBPF edge** allowlists, groups/CNP, Mbps/PPS, and flows (FluxVM Network Fabric schema v4), see VM Dataplane and Edge Dataplane — that plane is orthogonal and does not replace these host policies. Maglev VIP load balancing is Service Fabric v6 (BPF schema 4) on the same Edge Dataplane **Services** tab (ebpf-service-fabric.md).

When to use it

- To write a network policy scoped to a security identity (a label-based group of VMs) rather than to individual IPs
- To manage host firewall zones/profiles and see which VMs are assigned to which firewall profile
- To set up a WireGuard VPN tunnel or network between sites
- To rate-limit a VM's traffic with a QoS policy, or mirror its traffic to a collector for inspection
- To bring firewalld/nftables/WireGuard configuration that already exists on the host under Zyvor's management instead of recreating it by hand

How to get there

- Route / id: `/app/network-security`
- Nav: **Infrastructure → Net Security** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Nine tabs, each with its own live count in the header: **Policies, Firewall, Services, QoS, DNS, VPN, Mirror, NAT, Monitor**. Data refreshes automatically every 15 seconds and on demand via **Refresh**. Read-only accounts see a read-only notice and can't create, adopt, sync, or delete anything.

1. **Policies** — network policies bound to security identities (identities are auto-created from VM labels or discovered from firewalld zones/nftables sets on the host). Create or delete a policy; adopt a discovered policy or identity into management.
2. **Firewall** — firewall profiles, zones, and per-VM assignments. Create/delete profiles and zones, remove a VM's firewall assignment, and adopt a zone or profile that already exists on the host.
3. **Services** — exposed host listeners. Create/delete a service, or adopt one already running on the host.
4. **QoS** — bandwidth-shaping policies. Create/delete/adopt.
5. **DNS** — DNS zones and DNS policies. Create/delete/adopt both zones and policies.
6. **VPN** — WireGuard tunnels and VPN networks. Create/delete tunnels and networks; adopt a tunnel already configured on the host.
7. **Mirror** — traffic mirror sessions that copy a VM's traffic to a collector address. Create/delete/adopt.
8. **NAT** — NAT rules, NAT pools, and gateway configs. Create/delete each; adopt a NAT rule found on the host.
9. **Monitor** — bandwidth monitor policies, live traffic metrics, and bandwidth alerts. Create/delete/adopt a monitor policy, and acknowledge an alert.

Every tab has a **Sync** action that rescans the host's actual configuration (firewalld, nftables, WireGuard, etc.) for items not yet under Zyvor's management — synced items show up as adoptable so you can bring them under management with one click instead of recreating them.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- VM Dataplane — FluxVM eBPF edge (orthogonal)
- Network — NAT / bridge / port forwards
- Getting Started
- Page index

Network

Purpose

Network — day-to-day VM networking: which mode a VM uses, its port forwards, and its assigned address. For the advanced SDN stack (policies, firewalls, VPN mesh, QoS, mirroring), see Net Security. For per-VM TC/eBPF edge allowlists, rate limits, and flows, see VM Dataplane.

When to use it

- To see how a VM is networked and what's reachable from outside the host
- To add or remove a port forward without recreating the VM
- To check a VM's assigned IP address
- To jump into **Dataplane** for eBPF edge policy on a bridged VM

How to get there

- Route / id: `/app/network`
- Nav: **Infrastructure → Network** (sidebar, command palette, or desktop nav)
- Per-VM networking is also managed from that VM's own **Network** tab on its detail page (includes **Open Dataplane**)

Operate from the console (UX)

1. Review VMs by networking mode:
   - **NAT** — private, outbound-only by default; individual ports can be exposed with a host→guest port forward (set at creation in the Create VM wizard, or added later from a VM's Network tab).
   - **Bridged** — the VM has its own address on the local network, either DHCP-assigned or set statically via cloud-init at boot. Bridged VMs are what attach FluxVM Network Fabric eBPF.
2. Add or remove a port forward for a running or stopped VM — if the VM is running, it restarts automatically to apply the change.
3. Confirm a forwarded port is actually reachable from another machine on your network, not just from the host itself.
4. Check a bridged VM's assigned IP address once it's leased (or immediately, if it was assigned statically).
5. Open **Dataplane** from the Network teaser when you need edge allowlists / Mbps caps / flows.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- VM Dataplane
- Net Security
- Create VM
- Virtual Machines
- Getting Started
- Page index

Resource Pools

Purpose

Resource Pools — hierarchical CPU/memory allocation pools (nested, with shares, reservations, and limits) that VMs draw from, plus an admission-control test to check whether a workload's requirements would fit before you commit to it.

When to use it

- To divide a cluster's CPU/memory into weighted pools — for example giving "prod" more shares than "dev"
- To nest a child pool under a parent to further subdivide its allocation
- To check whether a pool has room for a new VM's CPU/memory requirements before creating it
- To see how much of a pool's reservation or limit is actually in use

How to get there

- Route / id: `/resource-pools`
- Nav: **Infrastructure → Resource Pools** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review stat cards: total pools, total CPU shares, and total VMs across all pools.
2. Browse the expandable pool tree — click a pool to expand/collapse its children; each row shows VM count, CPU/memory shares, and live usage bars that turn red above 80%.
3. Expand a pool to see its CPU/memory reservation, limit, currently available capacity, and child pool count.
4. **Create Pool** — set a name, cluster ID, an optional parent pool (to nest it under another pool), and CPU/memory shares, reservations, and limits (use `-1` for unlimited).
5. **Test Admission** — on any pool, enter a required CPU (MHz) and memory (MB) and run the check; the result shows Admitted or Denied with a reason and the pool's currently available capacity.
6. Delete a pool — asks for confirmation before removing it.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Network
- Net Security
- Edge Dataplane
- Getting Started
- Page index

Storage Pools

Purpose

Storage Pools — create, start/stop, and monitor the storage backends VM disks live on: local directories, NFS exports, LVM and LVM-thin volume groups, ZFS pools, or Ceph RBD pools. This is where a pool's lifecycle and health live; for the volumes (disks) inside those pools see Storage, and for multi-host replicated/policy-driven storage see Distributed Storage.

When to use it

- To add a new storage backend — e.g. mount an NFS export or attach a Ceph RBD pool — for VM disks to be created on
- To start or stop a pool, or delete one that's no longer needed
- To check NFS or Ceph pool health at a glance
- To search for a pool by name, path, or state and see its live capacity/usage

How to get there

- Route / id: `/app/storage-pools`
- Nav: **Infrastructure → Storage Pools** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review stat cards: total pools, active pools, total capacity, and total available space.
2. Search pools by name, path, or state.
3. Table of pools — name, type with backend-specific detail (NFS server:export path, LVM volume group, LVM-thin volume group/thin pool, ZFS zpool/dataset, or Ceph pool name + monitor count), path, capacity, available space, a usage bar, current state, and a health indicator for NFS/Ceph pools.
4. **Create Pool** — pick a type (Local, NFS, LVM, LVM-thin, ZFS, Ceph) and fill in its backend-specific fields (for NFS: server, export path, mount path, NFS version, mount options; for Ceph: monitor addresses, pool name, optional user/keyring; etc.), plus an auto-start-on-daemon-boot toggle.
5. Per-pool actions: **Start** an inactive pool, **Stop** an active one, **Refresh** to pull current stats, or **Delete** — disabled while the pool is active, so stop it first — with a confirmation dialog.
6. For Ceph pools, a **Manage RBD images** action opens a panel to list, create, and delete raw RBD images in that pool. This is proxied through the Atlas storage control plane (Zyvor's Ceph system of record) and requires Atlas to be configured on the backend; image creation is queued and completes asynchronously rather than immediately.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed. Ceph pool health/stats and RBD image management additionally require an Atlas connection — if that's not configured, those specific actions will show an unavailable error rather than the whole page failing.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Network

- Net Security
- Edge Dataplane
- Getting Started
- Page index

Storage

Purpose

Storage — a consolidated view of every storage pool's capacity alongside a manual volume tracking ledger. To create or manage a pool itself, use Storage Pools; for replicated/policy-driven storage across hosts, see Distributed Storage.

Important: the Volumes table is a **manual tracking ledger**, not live disk provisioning. Resize/Attach/Detach/Delete here update the record only — they do not change real disks or VM configs.

When to use it

- To see total capacity, used space, and pool count across the whole host in one place
- To keep a manual record of volumes you've provisioned elsewhere (pool, size, intended VM)
- Prefer this page when the job matches the purpose above
- Start from the Dashboard if you are unsure where to begin
- Before creating large VMs, to see which pools are near full

How to get there

- Route / id: `/app/storage`
- Nav: **Infrastructure → Storage** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review stat cards: total capacity, used space, volume count, and pool count.
2. Storage Pools panel — name, path, type badge, state, used/total bar, **Add Volume Record**.
3. Volumes table — name, pool, size, attached VM (or "Not attached"), plus **Resize**, **Attach/Detach**, **Delete** (confirmation).
4. Use records as notes for volumes you provision via Storage Pools (including Ceph RBD image management) or host tools.
5. Pool create/start/stop remains on Storage Pools; multi-host policy on Distributed Storage.

Typical flow: check pool fullness → add volume records for inventory → manage real pools on Storage Pools → use Storage Mgr for a read-only pool/volume browser if preferred.

Operator tip: if a pool bar is red, free space or expand on Storage Pools before creating large disks.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Storage Pools
- Distributed Storage
- Storage Mgr
- Capacity

System Health

Purpose

System Health — a live, read-only dashboard of host resource utilization, refreshing every 2 seconds: CPU, memory, disk I/O, filesystems, network interfaces, and top processes, rolled up into a single health score.

When to use it

- To get a fast, single-number (0–100) read on whether the host is under stress
- To find out what's bottlenecking the host — CPU, memory, disk, or network — before adding more VMs to it
- To identify the specific process consuming the most CPU or memory
- To check filesystem usage or per-disk I/O latency and queue depth on the physical host

How to get there

- Route / id: `/app/system-health`
- Nav: **Infrastructure → System Health** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Health score gauge (0–100, colored red/amber/green) with a status label and a summary panel calling out the current bottleneck, if any.
2. **CPU** panel — overall usage, core count, 1-minute and 5-minute load averages, and a per-core usage grid.
3. **Memory** panel — RAM and swap usage bars with used/total detail, plus total, available, and cached memory.
4. **Filesystems** — a usage bar per mounted filesystem (mountpoint, filesystem type, used/total), shown when data is available.
5. **Disk I/O** — per-device reads/writes completed, bytes read/written, average latency, and queue depth.
6. **Network** — per-interface RX/TX bytes and error counts, TCP connection-state counts, and host-wide RX/TX/retransmit totals.
7. **Top CPU** and **Top Memory** process tables — PID, name, CPU%, and memory (MB).

Entirely read-only monitoring — there's no create/edit/delete action here; data refreshes automatically every 2 seconds.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Network
- Net Security
- Edge Dataplane
- Getting Started
- Page index

System

Purpose

System — the physical host's hardware topology (CPU sockets/cores/threads, NUMA nodes, hugepages) plus topology-aware optimization recommendations for individual VMs. This is distinct from System Health, which tracks live utilization rather than hardware layout.

Four tabs: CPU Topology, NUMA Topology, Memory & Hugepages, Optimization (badged with pending recommendation count).

When to use it

- To see the host's CPU socket/core/thread layout and NUMA node boundaries before pinning a VM's vCPUs
- To check how many hugepages (2MB/1GB) are allocated and free, or allocate more for a memory-intensive VM
- To review and apply topology-aware optimization recommendations (e.g. NUMA/CPU pinning) for a specific running VM
- Prefer this page when the job matches the purpose above
- Before placing latency-sensitive guests, to understand NUMA distances

How to get there

- Route / id: `/app/system`
- Nav: **Infrastructure → System** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Stat cards: total CPUs (sockets × cores), NUMA node count, total/available memory, 2MB hugepage total/free.

1. **CPU Topology** — CPUs by socket, online/offline; hover for core, thread, NUMA node.
2. **NUMA Topology** — per-node CPUs, memory total/free + bar, hugepage counts, inter-node distance matrix.
3. **Memory & Hugepages** — memory breakdown (total, available, buffers, cached) plus 2MB/1GB hugepage stats. **Allocate Hugepages** picks page size and count and applies immediately.
4. **Optimization** — per-VM recommendations (resource, current → recommended, reason, impact). **Apply** runs the change against that VM.

Typical flow: learn topology → allocate hugepages if needed → open Optimization → Apply carefully on non-prod first → confirm on VM detail / Live Metrics. For live utilization (not layout), use System Health.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- System Health
- Resource Pools
- Optimizer
- Live Metrics
- Kernel

- Virtual Machines
- Getting Started
- Page index

Warm Pools (VM Pools)

Purpose

Warm Pools — keep N VMs **pre-booted and paused** from a chosen disk image so you can **claim** one instantly instead of cold-creating. Each pool shows ready members vs size, vCPUs/memory, and the image path.

In the UI and Core nav this is **Warm Pools** (`/app/vm-pools`). Older docs may say “VM Pools.”

When to use it

- To cut provision latency for bursty or CI-style workloads
- To keep a standing set of identical paused members ready to claim
- To claim a ready member into a named running VM and jump straight to its detail page
- To tear down a pool (and its members) when you no longer need the capacity

How to get there

- Route / id: `/app/vm-pools`
- Nav: **Core → Warm Pools** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. On first visit with no pools, the empty state explains the feature and offers **Create Pool**.
2. **Create Pool** — pool name, disk image (from catalog), size (member count), vCPUs, memory → submit. A success toast notes that members are booting; the ready count climbs as they pause.
3. Each pool card shows name, image path, **Ready to claim** `ready / size` with a progress bar, and vCPU/memory.
4. **Claim Instantly** — enabled only when `ready_members > 0`. Enter the VM name to assign; on success you navigate to `/app/vms/:name`.
5. **Delete** (trash) — confirmation warns that the pool and its pre-booted members will be removed; member teardown continues in the background after the pool disappears from the list.
6. **Empty / fail:** Load error banner with retry; create/claim/delete toasts on failure; Claim disabled when no ready members (tooltip: “No ready members right now”).
7. **Success:** Pool card appears with rising ready count; claim opens the new VM detail page with a success toast.

Related pages

- Create VM
- Virtual Machines
- Profiles
- Resource Pools
- Getting Started
- Page index

Availability Zones

Purpose

Zones — availability / placement domains for the fabric: named zones (region, status, host count) plus **spot instances** (max price, priority, eviction policy) that can be requested, evicted, or deleted against those zones.

In the UI this page is titled **Availability Zones**.

When to use it

- To define placement domains (e.g. `us-east-1a`) before you reason about spot capacity
- To see which zones are `available`, `degraded`, or `unavailable` and how many hosts each has
- To request a spot instance for a VM with a max price/hour and eviction policy
- To evict a running spot instance (applies its eviction policy immediately) or delete the spot record

How to get there

- Route / id: `/app/zones`
- Nav: **Operations → Availability Zones** (sidebar, command palette, or desktop nav)
- Legacy `/zones` redirects under `/app` when configured

Operate from the console (UX)

Summary tiles show zone count, running spot instances, and evicted count. Two tabs: **Zones** and **Spot Instances**. Use the header refresh to reload.

Zones tab

1. **Create Zone** — name (required), optional description and region → **Create**.
2. The table lists Name, Region, Status badge, Hosts count.
3. Delete a zone with the trash icon (confirmation required).
4. Empty state: “No availability zones.”

Spot Instances tab

1. **Request Spot Instance** — VM name, max price/hour, priority (`low` / `regular`), zone, eviction policy (`stop` / `delete` / `deallocate`).
2. Table columns: VM, Max Price/hr, Priority, Status (`running` / `evicted` / `terminated`), Eviction Policy.
3. On a **running** spot: **Evict** applies the eviction policy to the VM immediately (confirmation).
4. Trash deletes the spot **record** (confirmation), separate from evict.
5. Empty state: “No spot instances.”
6. **Empty / fail:** Page load banner “Could not load availability zones” — check auth and API; create/evict toasts on failure.
7. **Success:** New zone or spot row appears; evict moves status to `evicted` and shows a success toast.

Related pages

- Datacenters
- DRS
- Resource Pools

- Virtual Machines
- Getting Started
- Page index

Alerts

Purpose

Alerts — a live view of currently firing system alerts and the notification rules that generate them, polling for updates automatically.

Read-only here: you cannot acknowledge, mute, or edit rules on this page. Configure delivery on Notifications.

When to use it

- Prefer this page when the job matches the purpose above
- To see at a glance whether anything is critical or warning-level right now
- To check what an active alert actually means before deciding whether to act
- To review which alert rules are configured and enabled, and at what threshold they fire
- As a first stop during on-call before opening Timeline/Logs
- To confirm the fleet is clear ("No active alerts") after remediation

How to get there

- Route / id: `/app/alerts`
- Nav: **Monitoring → Alerts** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Active Alerts summary** — total active, critical count, warning count.
2. **Active Alerts list** — cards with severity badge, timestamp, title, message, triggering value when available; left border red/amber/blue by severity. Empty → "No active alerts".
3. **Alert Rules table** — when rules exist: name, condition, threshold, severity, enabled Yes/No (read-only).
4. Auto-refresh every 5 seconds; failed background refresh keeps last data with an amber note. Header refresh forces reload.

Typical flow: read critical cards → follow message to the subsystem (VM, storage, host) → remediate → wait for poll to clear. Wire Slack/email via Notifications if you need push delivery of the same signals.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Notifications
- Timeline
- Logs
- Audit
- Security Dashboard
- Live Metrics
- Getting Started

Analytics

Purpose

Performance Analytics — fleet-wide resource utilization, trends over time, and per-VM performance insights, with exportable reports.

Time range drives the chart and reloads page data. Read-only — act on VMs from their detail page.

When to use it

- To check overall CPU/memory/disk/network utilization across the fleet
- To find which VMs are consuming the most CPU, memory, or network right now
- To review flagged performance issues (e.g. a VM running hot) with a recommendation
- To pull a PDF or CSV performance report for a stakeholder or incident writeup
- For longer windows than Live Metrics' 60-second sparklines (up to 30 days)

How to get there

- Route / id: `/app/analytics`
- Nav: **Monitoring → Analytics** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Time range** — Last Hour, 6 Hours, 24 Hours, 7 Days, or 30 Days; reloads chart and data.
2. **Export** — PDF or CSV performance report for the selected range.
3. **Resource Utilization Overview** — CPU, Memory, Disk, Network tiles with % and color bars (green <50%, blue 50–75%, amber 75–90%, red 90%+).
4. **Performance Insights** — up to 5 issues (critical/warning/info) with VM, resource, value, recommendation.
5. **Top VMs by Resource** — top 5 by CPU, Memory, and Network.
6. **System Performance Over Time** — area chart of total CPU% and memory% plus averages and VM counts for the window.

Typical flow: pick 24 Hours or 7 Days → scan Insights and Top VMs → Export if needed → right-size via Optimizer or open the hot VM. Use Explain for a prose take on one metric.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Live Metrics
- Explain
- Optimizer
- Capacity
- Alerts
- Virtual Machines

Audit

Purpose

Audit Logs — the security and compliance trail of who did what: every tracked action, which user performed it, on which resource, whether it succeeded, and from what IP address.

Prefer Audit when you need **actor + IP + success/fail**. Prefer Logs for streaming console-style messages; Timeline for a merged narrative.

When to use it

- To investigate who created, deleted, started, or stopped a specific VM (or other resource) and when
- To check recent failures for signs of misconfiguration or unauthorized attempts
- To pull an export of the audit trail for a compliance review or incident report
- After access-control changes, to verify which admin performed them
- When correlating failed logins from Security Dashboard with resource actions

How to get there

- Route / id: `/app/audit`
- Nav: **Monitoring → Audit** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Stats row** — total logs, success rate, recent failures, top 3 most common actions.
2. **Search** — matches action, user, resource name, and resource type (Enter or live filter).
3. **Filters** — Status (success/failed) and Resource Type (VM, network, storage, template, quota, schedule); **Clear Filters** resets filters and search.
4. **Export** — downloads the filtered set as JSON or CSV.
5. **Logs table** — relative timestamp, user, action (color-coded), resource type/name, success/failed badge (error inline on fail), source IP. Footer shows filtered vs total counts.

Typical flow: filter Failed → search VM name → Export CSV/JSON for the ticket → open Access Control if a user must be disabled. Read-only history.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Logs
- Timeline
- Access Control
- Security Dashboard
- Compliance
- Notifications

Capacity

Purpose

Capacity Planning — resource usage against total capacity per resource (memory, CPU, storage), with week-over-week trend, so you can see what's running out and when.

Read-only reporting. A warning banner appears when any tracked resource is above 75% usage.

When to use it

- Prefer this page when the job matches the purpose above
- To check whether the host is approaching a resource ceiling before it becomes an incident
- To see which resources are growing week over week and by how much
- To get a rough sense of how many active VMs are contributing to current usage
- Before approving large template/profile creates or raising Quotas
- In weekly capacity reviews alongside Analytics trends

How to get there

- Route / id: `/capacity-planning`
- Nav: **Monitoring → Capacity** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Capacity warning banner** — appears when any tracked resource is above 75%, naming which ones.
2. **Summary tiles** — Active VMs, Resources Tracked, resources Over 75% Usage, Growing Resources.
3. **Per-resource cards** — used/total (GB→TB past 1024 GB), usage %, color progress bar, weekly trend (%/week with arrow), remaining capacity, projected-full date when supplied.
4. Auto-refresh every 30 seconds; header refresh forces immediate reload.

Typical flow: open Capacity → note Over 75% / Growing → open the hot resource card → plan reclaim (Optimizer, stop idle VMs) or expand storage via Storage Pools. No configure actions on this page.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Analytics
- Optimizer
- Quotas
- Storage
- Storage Pools
- Live Metrics
- Getting Started
- Page index

Debug Tools

Purpose

Debug Tools — raw, terminal-style output from four classic Linux diagnostic commands (top, iostat, vmstat, netstat) against the host, rendered as monospace panels. This is the closest the dashboard gets to SSHing in and running commands yourself.

Panels do **not** auto-load; click Refresh (per panel or Refresh All). Optional auto-refresh every 3 seconds.

When to use it

- Prefer this page when the job matches the purpose above
- To check live process/CPU activity, disk I/O, virtual memory stats, or network connections on the host without opening a shell
- To troubleshoot a performance problem in real time, side by side across all four views
- When Kernel or Analytics show something is wrong and you need the raw command output to confirm it
- When Processes is not enough and you want classic `top` / `vmstat` text
- To capture a pasteable snippet for an incident channel (copy from the monospace panels)

How to get there

- Route / id: `/debug`
- Nav: **Monitoring → Debug Tools** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Four independent panels** — Top, IOStat, VMStat, NetStat — each backed by its own endpoint, monospace scrollable output.
2. Each panel starts with "Click Refresh to load data" until you refresh that panel or use **Refresh All**.
3. **Auto-refresh** — header switch re-pulls all four every 3 seconds; turn off when done to stop polling.
4. A failed panel shows its own error without blocking the others.

Typical flow: Refresh All → enable Auto-refresh while reproducing → disable Auto-refresh → copy the hot panel output. Pair with Live Metrics for sparklines and Processes for PID drill-down. Read-only — no process control from here.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Processes
- Live Metrics
- Kernel
- Explain
- System Health

- Analytics
- Getting Started
- Page index

Event Stream

Purpose

Event Stream — a live, scrolling log of VM lifecycle events (create, start, stop, delete, and similar) pushed over an authenticated SSE connection as they happen. There's no history — you only see events that occur while the page is open.

"Waiting for events..." with a green Connected indicator is normal on a quiet fleet.

When to use it

- To watch VM lifecycle activity happen in real time, e.g. while running a script that creates or tears down several VMs
- To confirm an action you just took (start, stop, delete) actually registered
- To catch error/warning-level events as they occur without polling a log page
- During bulk ops or migrations when you want a live side channel
- When Logs is too noisy and you only care about lifecycle as it fires

How to get there

- Route / id: `/event-stream`
- Nav: **Monitoring → Event Stream** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Connection indicator** — Connected (green) or Reconnecting… (amber, pulsing) for the SSE link.
2. **Pause / Resume** — pause freezes the view and drops incoming events (does not queue); resume appends new events from that point.
3. **Clear** — empties the on-screen list (does not affect the connection).
4. **Level filter** — All, Info, Warning, Error, or Debug (level inferred from event type, e.g. names containing fail/error → Error).
5. Each line: time, level, source VM name, message. Keeps the most recent 500 events and auto-scrolls unless paused.

Typical flow: open stream → Connected → perform create/start elsewhere → watch the matching lines → Pause to inspect → Clear when starting a new test. For historical trails use Logs / Audit / Timeline.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Logs
- Timeline
- Audit
- Bulk Operations
- Virtual Machines

- Notifications
- Getting Started
- Page index

Explain

Purpose

Explain — plain-language, AI-generated explanations for a chosen system metric: its current value and trend, an assessment of its status, what's contributing to it, and what to do about it. This is the interpretive layer on top of the raw numbers you'd see in Analytics or Debug Tools.

Nothing loads until you pick a metric. The page never mutates the system — it only explains and recommends.

When to use it

- When a metric looks off and you want a written explanation of what's driving it, not just the raw number
- To get concrete recommendations for a specific resource (CPU, memory, disk, or network) instead of interpreting a chart yourself
- Before escalating an issue, to see whether the system already has a plausible explanation and fix
- After spotting a spike on Live Metrics or a flag in Analytics
- When sharing a short status narrative with someone who will not dig through charts

How to get there

- Route / id: `/explain`
- Nav: **Monitoring → Explain** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Pick a metric** — CPU, Memory, Disk, or Network. Selecting one fetches its explanation and last-hour timeseries.
2. **Current Value & Status** — current value, trend arrow (up/down/flat), status badge (e.g. normal, elevated, critical), short summary.
3. **Last Hour chart** — bar chart of samples; hover for exact value and time.
4. **Contributing Factors** — named factors with impact badge (high/medium/low) and short description.
5. **Recommendations** — checklist of suggested actions when the backend provides any.

Typical flow: pick the hot metric → read status + factors → follow recommendations manually on Virtual Machines / System / storage pages. Cross-check raw numbers on Analytics or Debug Tools before making invasive changes.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Analytics
- Live Metrics
- Debug Tools
- Optimizer
- Capacity

- Alerts
- Getting Started
- Page index

Kernel

Purpose

Kernel — a snapshot of the host's kernel configuration: version, hostname, architecture, boot command line, loaded kernel modules, and sysctl parameters. This is static configuration, not live activity — for that, see Debug Tools or Event Stream.

Read-only. Modules table is capped at the first 100 matching rows after filter.

When to use it

- To confirm what kernel version, architecture, or boot parameters the host is running
- To check whether a specific kernel module is loaded and what's using it
- To look up the current value of a sysctl parameter without shelling in
- Before enabling Network Fabric eBPF / dataplane features that depend on kernel capabilities
- When filing a support report and you need kernel version + boot cmdline in one place

How to get there

- Route / id: `/kernel`
- Nav: **Monitoring → Kernel** (sidebar, command palette, or desktop nav)
- Console: `http://127.0.0.1:<port>/kernel` or `https://<host>/kernel`

Operate from the console (UX)

1. **Summary tiles** — Kernel Version, Hostname, Architecture, Modules Loaded count.
2. **Boot Command Line** — shown verbatim in a code block when the backend reports one.
3. **Kernel Modules table** — name, size, and "used by" for each loaded module (first 100 matches). **Filter modules** narrows by name live.
4. **Sysctl Parameters table** — key/value pairs reported by the backend (no filter).
5. Auto-refresh every 10 seconds; header refresh forces immediate reload.

Typical flow: confirm version/arch → search modules (e.g. networking/storage-related) → note relevant sysctls → continue live troubleshooting in Debug Tools or Processes. You cannot load modules or change sysctls here.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Debug Tools
- Processes
- System
- System Health
- VM Dataplane
- Getting Started

Live Metrics

Purpose

Live Metrics — a real-time view of host performance: CPU, memory, disk I/O, and network throughput, each as a rolling sparkline that updates once per second.

Network RX/TX are **client-derived rates** from counter deltas, not raw totals. Pause freezes a spike for inspection.

When to use it

- Watching load in real time while you run a benchmark or try to reproduce a slow VM
- Confirming the host is actually under CPU, memory, disk, or network pressure right now
- Pausing the feed to freeze a spike so you can read the exact numbers or take a screenshot
- During a change window to watch impact as VMs start or migrate
- When Analytics trends look fine but you suspect a short spike

How to get there

- Route / id: `/app/live-metrics`
- Nav: **Monitoring → Live Metrics** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

- Six live cards — CPU Usage, Memory Usage, Disk Read, Disk Write, Network RX, Network TX — current value plus a 60-second sparkline, polled every second.
- Network RX/TX are computed rates (bytes/sec) from successive cumulative counter samples.
- **Pause / Resume** freezes or resumes the 1-second loop.
- Status dot: **Streaming** (green), **Paused** (amber), or **Error** (red).
- Background refresh failure after load → amber banner, last known values kept.
- First-load failure → error banner with retry hints instead of cards.

Typical flow: start Streaming → reproduce the issue → Pause on the spike → note which of the six cards peaked → follow up in Processes / Debug Tools / Explain.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Analytics
- Processes
- Debug Tools
- Explain
- System Health
- Capacity
- Getting Started

Logs

Purpose

Logs — a searchable, filterable console view of Zyvor Fabric's audit log: every recorded action and event, level-coded and continuously refreshed.

Polls every 5 seconds. **Clear** only clears the local view — underlying history returns on the next poll. Use **Export** for a filtered `.txt` slice.

When to use it

- Investigating what changed and when — VM creates/deletes, config changes, and other recorded actions
- Filtering down to just `ERROR` / `WARN` entries to find what went wrong
- Searching by keyword or source, or exporting a filtered slice, before opening a support ticket
- During live debugging with Auto-scroll on
- When Timeline showed an Error and you need the full message text

How to get there

- Route / id: `/app/logs`
- Nav: **Monitoring → Logs** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Filter box** — matches message and source as you type.
2. **Level dropdown** — `ALL`, `INFO`, `WARN`, `ERROR`, or `DEBUG`.
3. **Auto-scroll** — pins to newest entries as the feed polls every 5 seconds.
4. **Refresh** — manual reload in addition to the automatic poll.
5. **Export** — downloads currently filtered entries as `.txt` (timestamp, level, source, message).
6. **Clear** (trash) — clears the local view only; entries reappear on next refresh/poll.
7. Entries are color-coded by level and show timestamp, level, source, and message.

Typical flow: set Level to ERROR → search by VM name → Export the slice → open Audit if you need user/IP/resource columns for compliance.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Audit
- Timeline
- Alerts
- Notifications
- Event Stream
- Getting Started

Notifications

Purpose

Notifications — configure how and when Zyvor Fabric alerts you: the delivery channels (email, Slack, Teams, webhook), the rules that trigger them, and a history of what was actually sent.

When to use it

- Setting up a Slack, Teams, email, or webhook channel to receive alerts
- Checking whether a rule has fired, how often, and whether delivery succeeded or failed
- Testing a channel to confirm it's wired up correctly before relying on it
- Temporarily disabling a rule or channel without deleting its configuration

How to get there

- Route / id: `/notifications`
- Nav: **Monitoring → Notifications** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

The page has three tabs — **Channels**, **Rules**, **History** — loaded together on open.

Channels (default tab)

- **Add Channel** opens a form: name, type (Webhook, Slack, Teams, Email), and type-specific fields — a URL for Webhook/Slack/Teams, or SMTP host/port + from/to address for Email.
- **Test** sends a real test notification through that channel (disabled if the channel isn't enabled).
- **Delete** removes a channel after a confirmation prompt.

Rules

- Each rule shows enabled/disabled state, its description, up to 3 event-type tags (with a "+N more" overflow), how many times it has triggered, and when it last fired.
- The enable/disable toggle (power icon) flips a rule on or off immediately.
- **Delete** removes a rule after a confirmation prompt.
- **Create Rule** opens a form: name, description, VM event types (created, started, stopped, snapshot taken, hotplug, error, etc.), severity levels, and which channels to notify. Every enabled rule is evaluated in real time against actual VM events — a matching rule sends through each selected channel and updates its trigger count and last-fired time here.

History

- A table of everything that's actually been sent: timestamp, rule name, event type (severity-colored), affected VM (if any), channel, and delivery status (`SENT` / `FAILED`).

If the channels list fails to load, an error banner with retry appears; rules and history failing to load independently just show as empty rather than blocking the page.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Alerts
- Logs
- Audit
- Timeline
- Webhooks
- Notification Center
- Getting Started
- Page index

Processes

Purpose

Processes — a live process monitor for the host: every OS process with its CPU and memory usage, refreshed every 3 seconds, with a per-process detail drill-down.

This is the **host** process table (not per-guest). For guest activity use VM Console or Live Metrics / Analytics.

When to use it

- Finding what's consuming CPU or memory on the host right now
- Spotting zombie or stopped processes, or an unexpectedly high process count
- Digging into a specific process's command line, I/O, open file descriptors, or context-switch activity
- When System Health shows pressure and you need the offending PID
- Alongside Debug Tools when you want a structured table instead of raw `top` text

How to get there

- Console URL pattern: `http://127.0.0.1:<port>/…` or `https://<host>/…`
- Route / id: `/processes`
- Nav: **Monitoring → Processes** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Summary cards: **Total Processes**, **Running**, **Sleeping**.
2. Filter box matches PID, process name, or state as you type.
3. Table: PID, name, CPU % (color bar — green <20%, blue <50%, amber <80%, red 80%+), memory MB, state badge (Running / Sleeping / Disk Wait / Zombie / Stopped), thread count.
4. Click a row for **Process Detail** — command line, IO read/write bytes, open FDs, voluntary/involuntary context switches. **Close** collapses it.
5. Auto-refresh every 3 seconds; manual refresh in the header.

Typical flow: sort attention to red CPU bars → open detail → note command line → correlate with FluxVM/QEMU or container workloads on Containers. Read-only — no kill/signal from this page.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Debug Tools
- System Health
- Live Metrics
- Containers
- Kernel
- Getting Started

Optimizer

Purpose

Optimizer — a right-sizing advisor that analyzes each VM's actual resource usage and recommends CPU/memory/disk adjustments, with a one-click apply per VM.

Recommendations are based on observed usage. **Auto-Optimize** applies that VM's recommendations via the optimize endpoint; skipped changes are reported in the result.

When to use it

- Checking whether VMs are over- or under-provisioned before a capacity or cost review
- Applying a recommended resource change without manually editing a VM's spec
- Triaging which VMs have high-impact recommendations first
- After a quiet period of metrics, so recommendations reflect steady state not a spike
- When Autoscale is too aggressive and you want a one-shot right-size instead

How to get there

- Route / id: `/resource-optimizer`
- Nav: **Monitoring → Optimizer** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Summary cards: **VMs Analyzed**, total **Recommendations**, **High Impact**, **Medium Impact**.
2. If nothing needs changing, the page shows "All VMs are optimally configured."
3. Otherwise each VM card lists recommendations: resource, current → recommended, reason, impact badge (High / Medium / Low).
4. **Auto-Optimize** per VM applies all of that VM's recommendations — button spins, then becomes disabled **Applied** and reports applied vs skipped counts.
5. Manual refresh is in the page header.

Typical flow: sort attention to High Impact → review reason → Auto-Optimize one VM → confirm on Virtual Machines / Live Metrics. Respect Quotas before applying large increases.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Analytics
- Capacity
- Live Metrics
- Autoscale
- Quotas
- Explain

Service Map

Purpose

Service Map — shows the services discovered across your VMs, which ones depend on each other (protocol and port), and each service's current health.

Use it to understand blast radius before restarting a VM or changing a service. Refreshes every 15 seconds.

When to use it

- Understanding what a VM's service depends on, or what depends on it, before restarting or changing it
- Spotting degraded or down services at a glance
- Tracing a specific inbound or outbound connection between two services
- During incident triage when you know a port/protocol but not the owning service
- After deploy, to confirm new services appear and dependencies look sane

How to get there

- Console URL pattern: `http://127.0.0.1:<port>/…` or `https://<host>/…`
- Route / id: `/service-map`
- Nav: **Monitoring → Service Map** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Summary cards show **Services**, **Healthy**, **Degraded**, and **Down** counts.
2. Service cards show name, health dot (green/amber/red/gray), type badge (web, database, cache, queue, api, proxy, …), host VM, port, and inbound/outbound link counts.
3. Click a service card to filter **Dependencies** to that service's links (both directions); unrelated cards dim. **Show all** clears the selection.
4. The Dependencies panel lists each link as `From → To` with protocol and port.
5. Auto-refresh every 15 seconds; manual refresh is in the page header.

Typical flow: find Down/Degraded cards → select one → read Dependencies → open the host VM from Virtual Machines if you need console or restart. This page is read-only discovery — it does not change networking.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Live Metrics
- Alerts
- Network Topology
- Virtual Machines
- Event Stream
- Getting Started

Timeline

Purpose

Timeline — a single reverse-chronological activity feed that merges audit-log actions and system alerts, so you can see what happened and in what order without switching between Logs and Notifications.

Client-side filter chips classify entries; the feed auto-refreshes every 10 seconds.

When to use it

- Prefer this page when the job matches the purpose above
- Reconstructing a sequence of events — what led up to an error, in order
- Getting a quick "what's happened lately" view of the whole system
- Filtering down to just deploys, or just errors, to review one class of events
- During an incident, before diving into raw Logs or Audit
- After a change window, to confirm expected Actions/Deploys and no unexpected Errors

How to get there

- Console URL pattern: `http://127.0.0.1:<port>/…` or `https://<host>/…`
- Route / id: `/timeline`
- Nav: **Monitoring → Timeline** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Filter chips — **All**, **Actions**, **Alerts**, **Deploys**, **Errors** — filter the merged feed client-side.
2. Each entry is auto-classified with icon/color, description, relative timestamp, and type tag: failed/error audit → **Error**; create/deploy-style → **Deploy**; other audit → **Action**; alerts → **Alert** (or **Error** if severity is critical/error).
3. The feed auto-refreshes every 10 seconds; the header shows "Updated Xm ago," plus a manual refresh button.
4. If a background refresh fails after data has already loaded, an amber banner reports it while the last known feed stays on screen.

Typical flow: open Timeline → filter **Errors** → note timestamps → jump to Logs/Audit for the same window → clear filter to **All** for surrounding context. This page does not acknowledge or mute alerts.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Logs
- Audit
- Alerts
- Notifications
- Event Stream

Backup Scheduler

Purpose

Backup Scheduler — create and manage recurring, automated backup jobs that snapshot one or more VMs' disks to a directory on a schedule, with retention and format controls.

Companion to one-off Backups. Schedules land under a host directory (default `/var/lib/zyvor-fabricd/backups`).

When to use it

- To set up nightly or weekly backups for a group of VMs instead of exporting disks by hand
- To back up several VMs on a shared cadence (e.g. every 6 hours) with a single schedule
- To control how much backup history is kept, and in what disk format, without babysitting the job
- Prefer this page when the job matches the purpose above
- When Operations → Backups covers ad-hoc jobs but you need recurrence + retention

How to get there

- Route / id: `/backup-scheduler`
- Nav: **More — images, migrations & managers → Backup Scheduler** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Select VMs — check VMs (state shown); **Select All** / **Deselect All**.

Schedule Configuration

- **Schedule Name** (e.g. `nightly-backup`)
- **Frequency** — Daily 2 AM, Weekly Sun 3 AM, Every 6h, or Custom cron (plain-English readout)
- **Output Directory** (default `/var/lib/zyvor-fabricd/backups`)
- **Retention (keep last N)** — 1–365
- **Format** — QCOW2, RAW, or VMDK
- **Enable compression** toggle

Create Schedule requires ≥1 VM and a name. **Existing Schedules** lists enabled/disabled, cron (raw + human), VM count, next run when available.

Typical flow: select critical VMs → Daily 2 AM + retention → Create → confirm next run → restore still via Operations → Backups when needed. Watch jobs on Job Monitor if a run fails.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Backups
- Snapshots

- Snapshot Mgr
- Schedules
- Replication
- Job Monitor
- Pipeline
- Getting Started
- Page index

Batch Import

Purpose

Batch Import — bulk-create VMs from a YAML or JSON list, with a preview step and a per-VM status readout as each one is submitted.

Creates VMs via the VM creation API one at a time after Preview. Use [Download Template] for a three-VM sample file.

When to use it

- To stand up several VMs at once from a single definition file instead of repeating Create VM
- To reprovision from a saved YAML/JSON inventory starting from the downloadable template
- To spot bad entries (missing name or image) before anything is created
- Prefer this page when the job matches the purpose above
- When Batch Migration only builds a migration spec and you need actual creates

How to get there

- Route / id: `/batch-import`
- Nav: **More — images, migrations & managers → Batch Import** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Provide input** — drop `.yaml` / `.yml` / `.json`, browse, or paste. Each entry needs `name` and `image`; `cpus` / `memory` optional (default `2` / `2G`). **Download Template** for examples.
2. **Preview** — parses into a table; missing name/image → inline error.
3. **Review** — name, vCPUs, memory, image path, status icon (pending/submitting/submitted/failed).
4. **Submit All** — creates sequentially; live row status; failures show under the image path; summary tracks total/submitted/failed.
5. **Back to Editor** — fix and re-preview without losing place.

Typical flow: Download Template → edit names/images → Preview → Submit All → check Virtual Machines / Event Stream. Respect Quotas before large batches.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Create VM
- Manifest Builder
- Batch Migration
- Quotas
- Virtual Machines
- Job Monitor

- Pipeline
- Getting Started
- Page index

Batch Migration

Purpose

Batch Migration Builder — a form-based editor for assembling a multi-VM migration job spec (source disk, target format, sizing) and exporting it as JSON. It builds the spec only; it does **not** run the migration itself.

Export with **Copy** or **Download** (`batch-migration.json`) for tools/pipelines elsewhere.

When to use it

- To describe several VMs' migrations (source path, target format, vCPUs, memory) before handing off
- To generate a JSON migration manifest to copy, save, or version-control
- To sketch VMDK→QCOW2 conversions without a text editor
- Prefer this page when the job matches the purpose above
- When you want a spec first; run live imports later via Migration Wizard / pipelines

How to get there

- Route / id: `/batch-migration`
- Nav: **More — images, migrations & managers → Batch Migration** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Add VM** — adds a numbered, expanded entry card.
2. Fill **VM Name**, **Source Path** (e.g. `/path/to/disk.vmdk`), **Target Format** (QCOW2/RAW/VMDK), **vCPUs**, **Memory (MB)**. Collapse headers to scan name/source.
3. Trash icon removes an entry.
4. **JSON Preview** builds `migrations: [...]` with `vm_name` , `source_path` , `target_format` , `cpus` , `memory_mb` once ≥1 entry exists.
5. **Copy** (clipboard confirmation) or **Download** as `batch-migration.json` .

Typical flow: Add entries → preview JSON → Download → feed your runner → watch Pipeline / Migration History. For a single guided import that actually runs, use Migration Wizard.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Migration Wizard
- Migration Templates
- Migration Readiness
- Migration History
- Job Monitor
- Pipeline

Disk Converter

Purpose

Disk Format Converter — convert a single disk image between QCOW2, VMDK, VHD, VHDX, and RAW, tracking the conversion job's progress to completion.

Converts one disk; does not create a VM. For import+VM create, use Migration Wizard.

When to use it

- To convert an imported disk (e.g. a VMDK) into QCOW2 before using it with a VM
- To produce a RAW or VHD/VHDX copy for a tool or target platform that needs a specific format
- To watch a long-running conversion through to completion without leaving the page
- Prefer this page when the job matches the purpose above
- After Upload Disk when the uploaded format is wrong for FluxVM

How to get there

- Route / id: `/disk-converter`
- Nav: **More — images, migrations & managers → Disk Converter** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Pick a source disk** — dropdown of known images or type a path (e.g. `/path/to/disk.vmdk`). Use **Load available disk images** if the list is empty.
2. **Choose the target format** — QCOW2, VMDK, VHD, VHDX, or RAW.
3. **Output path** — auto-derived from source + format; edit if needed.
4. **Convert** — submits and polls every 2 seconds; progress through pending → running → completed/failed.
5. Failure shows backend error inline; success shows output path. **Reset** clears form/job state.

Typical flow: load images → convert to QCOW2 → watch Job Monitor/Pipeline → download or attach the output. Batch JSON specs without running live on Batch Migration.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Disk Images
- Upload Disk
- Migration Wizard
- Batch Migration
- Job Monitor
- Pipeline
- Getting Started
- Page index

Disk Images

Purpose

Disk Images — a read-only inventory of the VM disk images present on the host: name, format, size, and path for each one.

Selection only feeds the Selected counter — there is no bulk action on this page.

When to use it

- To inventory what disk files exist before create/import/convert
- To search by name, format, or path and see total size across images
- Prefer this page when the job matches the purpose above
- When deciding whether to convert (VMDK→QCOW2) or upload a missing image

How to get there

- Route / id: `/disk-images`
- Nav: **More — images, migrations & managers → Disk Images** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review summary tiles: **Total Images**, **Total Size**, distinct **Formats**, and **Selected** count.
2. **Search** by name, format, or path.
3. Click a row or checkbox to select/deselect — local only; no bulk action attached.
4. Each row: name, color-coded format badge (qcow2, vmdk, vhd/vhdx, raw, img), size, full path.
5. **Refresh** reloads from the host.

Typical flow: search for a format → note path → open Disk Converter or Migration Wizard with that path → refresh after jobs complete. ISOs live on ISO Images.

Operator tip: Selected count is informational only — convert or download from their dedicated pages.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- ISO Images
- Upload Disk
- Disk Converter
- Storage Mgr
- Job Monitor
- Pipeline
- Getting Started
- Page index

Download Disk

Purpose

Download Disk — browse the disk images available on the Fabric host and download any of them straight to your machine.

Read inventory + download. Uploading is the inverse flow on Upload Disk.

When to use it

- To pull a host-side disk image onto your workstation for offline inspection or archive
- To retrieve a converted or migrated output path after a job finishes
- Prefer this page when the job matches the purpose above
- After Disk Converter or Pipeline completes, to fetch the output file

How to get there

- Route / id: `/download-disk`
- Nav: **More — images, migrations & managers → Download Disk** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Open the page and wait for the host disk-image list to load (or use header **Refresh**).
2. Browse or search the listed images — name, format, size, and path.
3. Choose an image and start the download to your browser's download location.
4. Confirm the file size matches expectations before transferring elsewhere.
5. For inventory without download, Disk Images is a read-only table of the same class of artifacts.

Typical flow: finish convert/migrate → open Download Disk → fetch the output path → optionally delete local copies when done. Paths are host-local under directories such as `/var/lib/…` — not remote lab URLs.

Operator tip: large downloads can take time; keep the tab open until the browser finishes.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Upload Disk
- Disk Images
- Disk Converter
- Migration History
- Job Monitor
- Pipeline
- Getting Started
- Page index

Image Builder

Purpose

Image Builder — build custom VM disk images from scratch using mkosi, by picking a Linux distribution and a package list, and track builds from queued through to a finished image.

Builds run as jobs; watch status through to a finished disk image usable for create/import.

When to use it

- To build a custom guest image with a chosen distro and package set
- To track build progress from queued to finished without shelling into mkosi manually
- Prefer this page when the job matches the purpose above
- When stock cloud images are not enough and you need a repeatable package baseline

How to get there

- Route / id: `/image-builder`
- Nav: **More — images, migrations & managers → Image Builder** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Choose a Linux distribution and the package list for the image.
2. Submit the build and note the job enters a queued/running state.
3. Track progress until the build completes or fails; capture the output image location on success.
4. Use header refresh / job views if the status looks stale.
5. After success, confirm the artifact on Disk Images and create a VM from it.

Typical flow: pick distro + packages → build → watch Job Monitor → register/use the image path. For one-off format conversion of an existing disk, use Disk Converter instead.

Operator tip: long builds belong in Job Monitor; do not assume a silent page means success.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Disk Images
- Create VM
- Content Library
- Upload Disk
- Job Monitor
- Pipeline
- Getting Started
- Page index

ISO Images

Purpose

ISO Images — a read-only inventory of installer and driver ISO files sitting in the host's configured images directory, showing which VMs currently have each one attached.

Read-only. Attachment changes happen from VM create/detail flows, not here.

When to use it

- To see which installer/driver ISOs are available on the host
- To find which VMs currently have a given ISO attached
- Prefer this page when the job matches the purpose above
- Before a guest install, to confirm the ISO path exists

How to get there

- Route / id: `/iso-images`
- Nav: **More — images, migrations & managers → ISO Images** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Open the page and load the ISO inventory from the configured images directory.
2. Review each ISO's identity and which VMs currently have it attached.
3. Use search/filter controls if present to narrow by name.
4. **Refresh** after copying a new ISO onto the host outside the UI.
5. Proceed to Create VM / VM detail to attach media — this page does not attach or detach.

Typical flow: confirm ISO present → create/boot VM with that media → return here to verify attachment listing. Disk files (not ISOs) are on Disk Images.

Operator tip: after copying an ISO onto the host, Refresh before expecting it in Create VM media pickers.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Disk Images
- Content Library
- Create VM
- Upload Disk
- Job Monitor
- Pipeline
- Getting Started
- Page index

Job Monitor

Purpose

Job Monitor — a live view of background jobs (disk conversions, migrations, and other pipeline work), with per-job progress, pipeline stage, and streaming logs.

Use this as the operator console for long-running image/migration work started elsewhere.

When to use it

- To watch conversion/migration jobs with progress, stage, and logs
- To diagnose a failed job from its streaming log without SSHing in
- Prefer this page when the job matches the purpose above
- After starting Disk Converter, Migration Wizard, Image Builder, or related pipelines

How to get there

- Route / id: `/job-monitor`
- Nav: **More — images, migrations & managers → Job Monitor** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Open Job Monitor to load active/recent background jobs.
2. Select a job to inspect percent complete, pipeline stage, and status.
3. Follow streaming logs for the selected job while it runs.
4. Refresh if the list looks stale after submitting work from another page.
5. On failure, capture the log excerpt, then retry from the originating tool (converter/wizard/builder).

Typical flow: start a conversion/migration → open Job Monitor → watch stage + logs → on success confirm Disk Images / Migration History. Pipeline is a stage-centric companion view.

Operator tip: keep this tab open during long conversions; stage stalls often show up in the streaming log first.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Pipeline
- Disk Converter
- Migration Wizard
- Migration History
- Image Builder
- Job Monitor
- Getting Started
- Page index

Manifest Builder

Purpose

Manifest Builder — a client-side form for assembling a VM configuration manifest and exporting it as YAML, with a live preview as you type. It doesn't create a VM or call the API; it's a scratchpad for drafting config to copy elsewhere.

No server mutation. Pair with Batch Import or API Playground when you are ready to apply.

When to use it

- To draft a VM YAML manifest with live preview before pasting into automation
- To explore fields without risking a create call
- Prefer this page when the job matches the purpose above
- When teaching teammates the shape of a VM config without using production APIs

How to get there

- Route / id: `/manifest-builder`
- Nav: **More — images, migrations & managers → Manifest Builder** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Fill the form fields for the VM configuration you want to express.
2. Watch the **live YAML preview** update as you type.
3. Copy or export the YAML when it looks right.
4. Apply elsewhere (Batch Import, API, git) — this page never submits creates.
5. Start over by clearing fields if the draft goes wrong.

Typical flow: draft → copy YAML → paste into Batch Import or an external pipeline → verify on Virtual Machines. For sizing presets already in the product, also see Profiles / Templates.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Batch Import
- Create VM
- API Playground
- Templates
- Profiles
- Job Monitor
- Pipeline
- Getting Started

History

Purpose

Migration History — a read-only log of completed and failed migration jobs, with status, timing, and where the output landed.

No filters/search/actions — a plain historical log. Empty → "No migration history yet."

When to use it

- To check whether a past migration succeeded or failed
- To read the error message left behind by a failed migration
- To see how long a migration took, or where its output disk was written
- Prefer this page when the job matches the purpose above
- After Pipeline clears, to confirm final status and output path

How to get there

- Route / id: `/migration-history`
- Nav: **More — images, migrations & managers → History** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. History loads from the migrations API on open; header **Refresh** reloads.
2. Columns: **Name**, **VM**, **Status** (completed / failed / running, with inline error under failed), **Started**, **Duration**, **Output** path.
3. No per-row actions — use output path with Download Disk / Disk Images as needed.
4. Correlate failures with Migration Report totals and Job Monitor logs.
5. For pre-checks before the next run, open Migration Readiness.

Typical flow: refresh after a batch → note failed rows + errors → fix source/path → re-run wizard → confirm completed + output path.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Migration Report
- Migration Readiness
- Migration Wizard
- Download Disk
- Job Monitor
- Pipeline
- Getting Started
- Page index

Readiness

Purpose

Migration Readiness — pre-flight checks that verify the environment is in a good state before you start a migration, with a pass/fail summary and per-check detail.

Run this before large Migration Wizard / batch work so failures are environmental, not mid-job surprises.

When to use it

- Before starting migrations, to verify host/environment readiness
- To read per-check pass/fail detail when the summary is not all green
- Prefer this page when the job matches the purpose above
- After fixing storage/network/FluxVM issues, to re-validate before retry

How to get there

- Route / id: `/migration-readiness`
- Nav: **More — images, migrations & managers → Readiness** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Open Readiness and run/load the pre-flight check set.
2. Read the pass/fail summary first.
3. Expand or scan per-check detail for anything failing.
4. Remediate owning systems (storage space, FluxVM health, network reachability to remote sources).
5. Re-run checks until the summary is clean, then proceed to Migration Wizard / Batch Migration.

Typical flow: Readiness → fix fails → Readiness again → Migration Wizard → Pipeline/History. Dashboard capability chips should already be Live before you trust a green readiness result.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Migration Wizard
- Batch Migration
- Dashboard
- Migration History
- Storage
- Job Monitor
- Pipeline
- Getting Started
- Page index

Report

Purpose

Migration Report — a shareable summary of all migration jobs (totals by status, average duration) plus the full per-migration detail table, with copy and print actions.

Use for stakeholder updates and post-mortems; History is the raw log, Report is the rollup.

When to use it

- To summarize migration outcomes (counts by status, average duration)
- To copy or print a shareable report after a migration wave
- Prefer this page when the job matches the purpose above
- After History shows the wave is done, to package results for others

How to get there

- Route / id: `/migration-report`
- Nav: **More — images, migrations & managers → Report** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Open Report to load totals by status and average duration.
2. Review the per-migration detail table alongside the summary.
3. Use **Copy** / **Print** (or equivalent share actions on the page) for handoff.
4. Refresh after new jobs complete so totals stay current.
5. Drill into failures via Migration History and Job Monitor logs.

Typical flow: finish wave → Report → copy/print → attach to change ticket → fix failures and re-run readiness/wizard as needed.

Operator tip: print/copy only after History shows the wave is idle so totals are not mid-flight.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Migration History
- Migration Readiness
- Pipeline
- Migration Wizard
- Job Monitor
- Getting Started
- Page index

Migration Templates

Purpose

Migration Templates — reusable migration configuration presets (disk format, vCPUs, memory, network, compression) that you can copy as JSON instead of re-entering the same settings for every migration.

Presets are copied as JSON for reuse; they do not by themselves execute a migration.

When to use it

- To standardize migration settings across many similar VMs
- To copy a preset as JSON into Batch Migration or an external runner
- Prefer this page when the job matches the purpose above
- When teams keep repeating the same format/CPU/memory/compression choices

How to get there

- Route / id: `/migration-templates`
- Nav: **More — images, migrations & managers → Migration Templates** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Browse existing migration presets (format, vCPUs, memory, network, compression).
2. Create or edit a preset when your standard settings change.
3. Copy the preset JSON for use in Batch Migration or other tools.
4. Apply the values in Migration Wizard / Batch Migration as appropriate.
5. Refresh the list after teammates add shared presets.

Typical flow: define a QCOW2 + sizing preset → copy JSON → paste into batch work → run readiness + wizard. For VM resource templates (not migration), see Operations → Templates.

Operator tip: treat copied JSON as a starting point — still run Readiness before a large wave.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Batch Migration
- Migration Wizard
- Templates
- Migration Readiness
- Job Monitor
- Pipeline
- Getting Started
- Page index

Network Topology

Purpose

Network Topology — a live map of which VMs are attached to which virtual networks or host bridges, alongside the host's own bridges and physical NICs, auto-refreshing every 15 seconds.

Read-only. Change attachments via Create VM or the VM's Network tab. For day-2 networking modes/port-forwards see Network; for Network Fabric schema v4 edge see VM Dataplane; Service Fabric v6 lives on Edge Dataplane.

When to use it

- To see which VMs share a network or bridge, or find VMs that aren't attached
- To check a host bridge's or physical NIC's operational state and addresses
- To look up a VM's interface type, MAC, and network without opening each VM
- Prefer this page when the job matches the purpose above
- During network incidents before changing Net Security policies

How to get there

- Route / id: `/network-topology`
- Nav: **More — images, migrations & managers → Network Topology** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Loads VM interfaces, virtual networks, host bridges, and links on open; refreshes every 15s; header **Refresh** forces reload.
2. Summary counts: virtual networks, host bridges, VMs.
3. **Host interfaces** — bridge/NIC cards with operational state and addresses.
4. **VM → network connections** — table of VM name/state, network/bridge, interface type, MAC.
5. **VM attachments** — per network/bridge cards listing attached VMs; unattached VMs grouped under "Unattached VMs."

Typical flow: find Unattached VMs → open VM Network tab to attach → Refresh topology → confirm. Do not confuse this map with eBPF dataplane policy (schema v4) or Maglev Services (Service Fabric v6).

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Network
- Net Security
- VM Dataplane
- Edge Dataplane
- Service Map

- Virtual Machines
- Getting Started
- Page index

Pipeline

Purpose

Pipeline Monitor — a live, auto-refreshing view of in-progress migration/conversion jobs, showing each job's percent complete and which of five stages it's currently in.

Stages: **inspect → prepare → convert → validate → deploy**. Read-only — no start/cancel/retry here.

When to use it

- To watch a migration or conversion in progress and see exactly which stage it's at
- To catch a failure as it happens instead of waiting for the history page to update
- To find a job's duration or output path as soon as it's available
- Prefer this page when the job matches the purpose above
- Alongside Job Monitor when you want a stage tracker more than raw logs

How to get there

- Route / id: `/pipeline`
- Nav: **More — images, migrations & managers → Pipeline** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Loads active jobs on open; polls every 3 seconds; header **Refresh** forces reload.
2. Each card: VM name, job ID, source, status badge (pending / running / completed / failed).
3. Progress bar shows percent complete.
4. Stage tracker: completed green, current pulses blue (red if failed), remaining grey.
5. Duration and output path appear when available; failures show a red error panel.
6. Read-only — start/cancel/retry from the tool that created the job.

Typical flow: submit Migration Wizard / converter → open Pipeline → watch stages through deploy → on failure read the error panel and Job Monitor logs → check Migration History.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Job Monitor
- Migration Wizard
- Disk Converter
- Migration History
- Migration Report
- Pipeline
- Getting Started
- Page index

Snapshot Mgr

Purpose

Snapshot Manager — create, revert to, and delete disk-state snapshots for a selected VM.

Sibling to Operations → Snapshots. Prefer Disk Only for routine checkpoints; Full (disk+memory) can take minutes under load.

When to use it

- Before a risky change, to capture disk state for rollback
- To revert a VM to an earlier snapshot after something went wrong
- To clean up old snapshots or check creation time and parent
- Prefer this page when the job matches the purpose above
- When you want a dropdown-scoped manager under More instead of `/app/snapshots`

How to get there

- Route / id: `/snapshot-manager`
- Nav: **More — images, migrations & managers → Snapshot Mgr** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Select VM** — dropdown loads snapshots; auto-refresh every 15s while selected, or **Refresh**.
2. **Create Snapshot** — name, **Disk Only** (default) or **Full (disk + memory)**; UI shows progress and retries if the monitor is still starting.
3. Table: Name, Created, State, Parent. **Revert** (confirmation; VM must be stopped for revert) and **Delete** (confirmation).
4. Empty states: no VM selected → "Select a VM"; none → "No snapshots."
5. Success banner confirms create.

Typical flow: select VM → Disk Only snapshot → make change → revert if needed (stop VM first) → delete old snaps. For fleet-wide timestamped snaps, Bulk Operations also offers Snapshot.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Snapshots
- Backups
- Backup Scheduler
- Bulk Operations
- Virtual Machines
- Job Monitor
- Pipeline

Storage Mgr

Purpose

Storage Manager — browse storage pools with their capacity and usage, and drill into a pool to see its volumes.

Read-only. Pool lifecycle lives on Storage Pools; the tracking ledger on Storage.

When to use it

- To check how full a storage pool is before provisioning new disks on it
- To see what volumes live in a given pool, and their format and size
- To find a volume's on-disk path
- Prefer this page when the job matches the purpose above
- When you want a quick pool→volumes browser under More

How to get there

- Route / id: `/storage-manager`
- Nav: **More — images, migrations & managers → Storage Mgr** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Pools load on open; header **Refresh** reloads.
2. Pool cards: name, type badge (dir / logical / netfs / disk / iscsi / rbd / zfs), state, usage bar (amber >70%, red >90%), used/capacity/available.
3. Click a pool to load its volumes.
4. Volumes table: Name, Format, Capacity, Allocation, Path.
5. No create/resize/delete here — use Storage Pools / Storage for management vs ledger.

Typical flow: scan red/amber pools → click pool → note volume paths → free space or expand via Storage Pools before new VMs. Capacity Planning gives host-level trends.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Storage
- Storage Pools
- Distributed Storage
- Disk Images
- Capacity
- Job Monitor
- Pipeline
- Getting Started
- Page index

Upload Disk

Purpose

Upload Disk Image — drag-and-drop (or browse) upload of a VM disk image file to the server, with live progress and an in-session upload history.

Accepted extensions: `.qcow2`, `.vmdk`, `.vhd`, `.vhdx`, `.raw`, `.img`, `.ova`. Upload history is **session-only** (clears on reload).

When to use it

- To get a disk image onto the host so it can be used to create a VM
- To upload a disk exported or converted elsewhere for import
- To check what you've uploaded recently in this session
- Prefer this page when the job matches the purpose above
- Before Migration Wizard or Create VM when the image is still on your laptop

How to get there

- Route / id: `/upload-disk`
- Nav: **More — images, migrations & managers → Upload Disk** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Drop zone accepts drag-and-drop or click-to-browse. Other extensions are rejected with an inline error.
2. Set **Destination Directory** (defaults to `/var/lib/libvirt/images`) and click **Upload**.
3. Live progress — percent, bytes transferred/total, speed — plus **Cancel**.
4. Success banner shows saved path, format, size; file appears in **Upload History** (name, format, size, time) for this session only.
5. Failure (format, network, cancel) shows an error banner.

Typical flow: upload → note path → convert with Disk Converter if needed → create/import VM. To pull images off the host, use Download Disk.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Download Disk
- Disk Images
- Disk Converter
- Migration Wizard
- Job Monitor
- Pipeline
- Getting Started

Autoscale

Purpose

Autoscale — define per-VM policies that automatically grow or shrink a VM's vCPUs and memory within set bounds based on load, and review the history of scaling actions that were triggered.

Policies are **per VM** (one policy per VM). Bounds and cooldown prevent runaway growth; recent scale events show what actually fired.

When to use it

- To let a VM's resources flex automatically instead of manually resizing it under load
- To cap how far a VM is allowed to scale (min/max vCPUs, min/max memory) so autoscaling can't run away
- To check what scaling actions actually fired and when, via the recent scale events log
- When a workload is bursty but still needs a hard ceiling for host capacity / quotas
- After reviewing Optimizer recommendations, if you want ongoing auto adjust instead of one-shot apply

How to get there

- Route / id: `/autoscale`
- Nav: **Operations → Autoscale** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review the policy table — one row per VM: vCPU range, memory range, CPU scale-up/scale-down thresholds, and cooldown (seconds between actions).
2. **Create Policy** — pick a VM (only VMs without an existing policy), set min/max vCPUs, min/max memory (MB), CPU scale-up and scale-down thresholds (%), and cooldown seconds.
3. **Delete** a policy from its row (confirmation) to stop autoscaling that VM.
4. Check **Recent scale events** — last 20 actions (VM, action, resource, timestamp).
5. On a read-only account, create/delete controls are hidden and a read-only notice is shown.

Typical flow: set conservative max bounds → create policy → watch Recent scale events under load → tighten thresholds or cooldown if it flaps. Confirm host capacity on Capacity and Quotas.

Scale events only appear after real threshold crossings; idle VMs with a policy and no load will show an empty recent log.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Quotas
- Optimizer
- Capacity
- Virtual Machines

- Schedules
- Getting Started
- Page index

Backups

Purpose

Backups & Restore — create full or incremental VM backups, track running backup/restore jobs, and restore a VM from a completed backup, either in place or as a new VM.

When to use it

- To take a one-off backup of a VM before a risky change, or as part of a regular backup routine
- To check on a backup or restore job that's currently running
- To roll a VM back to a previous backup, or clone it into a new VM from that backup

How to get there

- Route / id: `/app/backups`
- Nav: **Operations → Backups** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review the summary tiles: total backups, total size, a breakdown by type (full vs. incremental), and the timestamp of the newest backup.
2. Watch **Active Jobs** — running or queued backup/restore jobs with a live progress bar, refreshed automatically every 5 seconds.
3. **Create Backup** opens a dialog to pick a VM and choose **Full** or **Incremental**; the job is created compressed with a 30-day retention by default.
4. Search the backup table by VM name, type, or status, and review each backup's size, created time, expiry (or "Never"), and status (completed / in progress / failed).
5. **Restore** (enabled only once a backup is `completed`) opens a dialog where you either restore over the original VM, or check **Restore to new VM** and give it a name to clone the backup into a fresh VM instead. Restoring always brings back config and disks; VM runtime state is not restored.
6. **Delete** a backup permanently, after a confirmation dialog warning the action can't be undone.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Snapshots
- Backup Scheduler
- Replication
- Site Recovery
- Migrations
- Migration Wizard
- Schedules
- Getting Started

Bulk Operations

Purpose

Bulk Operations — select any number of VMs and start, stop, restart, or snapshot them together, with a per-VM progress log for the batch.

Actions run **sequentially** across the selection. There is **no confirmation dialog** — the action fires as soon as you click it. Prefer this over clicking one VM at a time on the fleet list.

When to use it

- To restart or stop a group of VMs at once (e.g. before a maintenance window) instead of one at a time
- To take a quick snapshot of several VMs together before a risky change
- To see which VMs in a batch action succeeded and which failed, with the specific error for each
- When Virtual Machines selection is awkward for a large filtered set
- For a maintenance window: filter → select → Stop or Snapshot → Clear results when done

How to get there

- Route / id: `/app/bulk-operations`
- Nav: **Operations → Bulk Operations** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Filter the VM list by name, then click rows to select (or **Select All** / **Deselect All** for the current filter).
2. Once at least one VM is selected, the action bar shows **Start**, **Stop**, **Restart**, and **Snapshot**. Snapshot creates `bulk-snap-<timestamp>` on each selected VM.
3. Each action runs sequentially; **Batch Progress** shows pending → running → done/error per VM, plus the error message when one fails while others succeed.
4. **Clear** deselects everything; **Clear results** dismisses the progress log after a run.
5. The table also shows each VM's state, CPU count, and memory while you select.

Operator tip: double-check the filter before **Select All** — there is no confirm step. For disk + memory snapshots of one VM with more control, use Snapshots or Snapshot Mgr.

Snapshot in bulk is disk-oriented via the bulk timestamp name; for Full (disk+memory) snapshots, use the per-VM Snapshots UI instead.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Virtual Machines
- Snapshots
- Schedules
- Backups

- Migrations
- Getting Started
- Page index

Content Library

Purpose

Content Library — a catalog of reusable provisioning building blocks: libraries of templates/ISOs/OVFs/scripts, guest customization specs (per-OS hostname/domain/DNS settings), and host compliance profiles.

Organize artifacts by **library** and path instead of scattering files across ad-hoc directories. Guest customization and host profiles live on the same page as separate tabs.

When to use it

- To organize VM templates, ISOs, OVF packages, and scripts into named libraries instead of scattering them across storage paths
- To define a reusable guest customization spec (hostname prefix, domain, DNS servers) for Linux or Windows guests
- To check host compliance profile status — how many hosts are compliant vs. non-compliant against a profile
- When onboarding a team that needs a shared ISO/template catalog
- Alongside Templates (resource presets) when you also need packaged media and guest identity specs

How to get there

- Route / id: `/app/content-library`
- Nav: **Operations → Content Library** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Summary tiles show total libraries, items, guest customization specs, and host profiles. Four tabs:

1. **Libraries** — cards with status, item count, and total size. **Create Library** sets name, optional description, **Local** or **Subscribed** type, and a required storage path. **Browse** jumps to Item Browser filtered to that library; trash deletes the library and all items (confirmation).
2. **Item Browser** — every item across libraries (template, ISO, OVF, script, or file) with type, version, size, last-updated. Filter by library dropdown or view all. Delete from the row (confirmation).
3. **Guest Customization** — specs (OS type, hostname prefix, domain, DNS). **Create Spec** sets name, Linux/Windows, hostname prefix, domain, and comma-separated DNS. Delete from the row (confirmation).
4. **Host Profiles** — compliant/non-compliant host counts and status. **Create Profile** sets name and optional description. Delete from the row (confirmation).

Typical flow: create a Local library with a storage path → add/browse items → create guest specs for naming/DNS → optionally track host profiles. For host patch baselines, also see Lifecycle.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Templates
- ISO Images

- Disk Images
- Lifecycle
- Compliance
- Create VM
- Getting Started
- Page index

DRS

Purpose

Distributed Resource Scheduler (DRS) — balances VM placement across the hosts in a cluster, surfaces migration recommendations, enforces affinity/anti-affinity rules, and can test where a new VM would land before you create it.

When to use it

- To see how evenly CPU and memory load is spread across your cluster's hosts
- To review (and approve or reject) migrations DRS suggests to rebalance load
- To keep certain VMs together (affinity) or apart (anti-affinity) — e.g. two replicas of the same service on different hosts
- To test where a hypothetical VM (given its vCPU/memory needs) would be placed before actually creating it

How to get there

- Route / id: `/app/drs`
- Nav: **Operations → DRS** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

DRS configuration (top of page) — toggle DRS **On/Off**, set the **Automation Level** (Manual, Semi-Auto, or Fully Auto), adjust the **Migration Threshold** (1–5, how aggressively it rebalances), and see the configured check interval. Changes apply immediately.

Four tabs below that:

1. **Balance** — the overall cluster balance score plus CPU/memory imbalance percentages, and a per-host breakdown with live CPU and memory usage bars (color-coded amber/red as usage climbs).
2. **Recommendations** — a table of migrations DRS suggests (VM, source host, target host, reason, priority, estimated benefit). Pending recommendations can be **approved** (checkmark) or **rejected** (X) directly from the row.
3. **Rules** — affinity/anti-affinity rules, each showing type, VM count, whether it's mandatory, and whether it's enabled. **Create Rule** sets a name, chooses **Affinity** (keep together) or **Anti-Affinity** (keep apart), lists VM IDs (comma-separated), and an optional mandatory flag. Delete a rule from its row (confirmation required).
4. **Placement** — a calculator: enter a hypothetical VM's CPU (MHz) and memory (MB), click **Test Placement**, and see the recommended host, its score and reasoning, projected CPU/memory usage after placement, and scored alternative hosts.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Snapshots
- Backups
- Migrations

Fault Tolerance

Purpose

Fault Tolerance (FT) — protect individual VMs with a live secondary replica on another host, so a host failure fails the VM over instead of taking it down, and monitor replication health.

When to use it

- To keep a critical VM running through a host failure by giving it a hot secondary on another host
- To check whether a VM is actually FT-compatible before enabling it, and fix any blocking issues first
- To test a failover safely, or trigger a real one when you need to move a VM off its current host
- To watch replication health — log bandwidth, checkpoint latency, secondary host load, uptime — for protected VMs

How to get there

- Route / id: `/app/fault-tolerance`
- Nav: **Operations → Fault Tolerance** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Summary tiles show protected VM count, how many are running, how many need attention, and total failover events. **Enable FT** opens a form to name the VM, set its primary host ID, and optionally a secondary host ID (leave blank to auto-select). Four tabs:

1. **Protected VMs** — table of FT-enabled VMs with their secondary host, status, and bandwidth limit. Per row: **Test** (a non-disruptive failover test), the play icon to **trigger a real failover** (confirmation required — moves the VM to its secondary host), and the trash icon to **disable FT** (confirmation required).
2. **Compatibility Check** — enter a VM ID and check whether it's FT-compatible; results show a pass/fail badge, any blocking or warning issues (each with a suggested fix), and recommended secondary hosts with a compatibility score.
3. **Events** — a timeline of failover events (test and real), each showing status, failover type, source → target host, downtime in ms, timestamp, and the error message if it failed.
4. **Metrics** — per protected VM: log bandwidth usage, checkpoint latency, secondary host's CPU/memory usage, failover count, and uptime percentage.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Snapshots
- Backups
- Migrations
- Getting Started
- Page index

Lifecycle

Purpose

Lifecycle Manager — define patch/upgrade baselines, scan hosts for compliance against them, remediate non-compliant hosts, and track rolling updates across a host fleet.

This is **host** patch/upgrade lifecycle, not VM create/start/stop. VM lifecycle actions stay on Virtual Machines, Schedules, and Bulk Operations.

When to use it

- To define a baseline (patch, upgrade, or extension) with a severity level, and see which hosts already meet it
- To scan hosts for compliance against a baseline and find out which are missing patches
- To watch a remediation task apply patches to a host, or a rolling update roll out across a fleet host by host
- Before a maintenance window, to know which hosts fail Critical/Important baselines
- After remediation, to confirm compliance scan results and rolling-update progress

How to get there

- Route / id: `/app/lifecycle`
- Nav: **Operations → Lifecycle** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Summary tiles: total baselines, non-compliant hosts, active remediation tasks, and running rolling updates. Four tabs:

1. **Baselines** — type, severity, release date, host count, compliant count, compliance bar. **Create Baseline** sets name, optional description, type (Patch/Upgrade/Extension), severity (Critical/Important/Moderate/Low). Play icon **runs a compliance scan**; trash **deletes** (confirmation).
2. **Compliance Scans** — per host: baseline, status (compliant / non-compliant / incompatible / etc.), missing patch count, last scanned time.
3. **Remediation** — tasks per host: status, progress bar, patches applied vs total, error message if any.
4. **Rolling Updates** — cards with status, hosts completed vs total, parallelism, current host, progress bar, timestamps, failed-host count.

Typical flow: create Critical patch baseline → run scan → review Compliance Scans → follow Remediation / Rolling Updates until non-compliant count drops. Pair with Compliance for config scorecards vs patch baselines.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Compliance
- Content Library
- System Health
- System

- Schedules
- Getting Started
- Page index

Migration Wizard

Purpose

Migration Wizard — a three-step wizard (Source → Configure → Review) for converting an existing disk image (local file or remote host) into a new Zyvor Fabric VM.

This imports/converts a **disk image into a new VM**. Live host-to-host moves of an existing Fabric VM live on Migrations. Spec-only batch builders live under More → Batch Migration.

When to use it

- To bring in a VM from elsewhere — a disk image on the local filesystem, or one reachable over SSH on a remote host
- To convert a disk image to a different format (QCOW2, RAW, or VMDK) as part of importing it
- To size the resulting VM's vCPUs and memory and decide whether it should auto-start once migration finishes
- When you already have a path like `/path/to/disk.vmdk` or `user@hostname:/path` and want a guided import
- After Migration Readiness checks pass for larger fleets

How to get there

- Route / id: `/migration-wizard`
- Nav: **Operations → Migration Wizard** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Source — choose **Local File** (disk image path, e.g. `/path/to/disk.vmdk`) or **Remote Host** (`user@hostname:/path` over SSH).

2. Configure — new VM name, target format (QCOW2, RAW, or VMDK), vCPUs (1–64), memory (MB, 256 MB steps), output directory (defaults to `/var/lib/zyvor-fabricd/images`), and whether to auto-start when done.

3. Review & Submit — summary of choices, then **Submit Migration**. Success or failure shows inline (with hints on failure); you can't resubmit once it succeeds. Use **Back** before submit to revisit earlier steps.

After submit, watch progress on Pipeline / Job Monitor, then confirm the VM on Virtual Machines. For format-only conversion without creating a VM, use Disk Converter.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Migrations
- Disk Converter
- Pipeline
- Job Monitor
- Migration History
- Backups
- Getting Started

Migrations

Purpose

VM Migrations — move a VM from its current host to a different target host, and track the migration from start to finish. The list auto-refreshes every 5 seconds so in-flight migrations update live.

When to use it

- To move a VM off a host for maintenance or to rebalance load
- To watch the progress of a migration currently in flight (percent complete, bytes transferred)
- To cancel a migration that's stuck or no longer needed
- To review past migrations — which VM, which target host, which type, and how they ended

How to get there

- Route / id: `/app/migrations`
- Nav: **Operations → Migrations** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Start Migration** — opens a dialog where you pick a VM from the dropdown (populated from your VM list), enter a target host (hostname or IP), and choose a migration type:
   - **Offline** — stop the VM, copy its data, then start it on the target
   - **Live** — migrate with minimal downtime
   - **Storage** — migrate storage volumes only
2. **Active Migrations** — each in-progress migration shows as a card with a live progress bar (percent complete), bytes transferred, its state badge (pending, precheck, syncing, switching), and a **Cancel** button. Any error from a failed migration is shown inline on the card.
3. **Migration History** — a table of completed, failed, and cancelled migrations showing VM, target host, migration type, final status, and start time.
4. **Refresh** — manually reload the list; the page also polls automatically every 5 seconds.
5. If there are no migrations at all, the page shows an empty state with a shortcut into the Start Migration dialog.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Snapshots
- Backups
- Migration Wizard
- Getting Started
- Page index

Quotas

Purpose

Resource Quotas — cap CPU, memory, disk, and VM-count usage, applied either globally or to VMs matching specific tags, so a team or workload can't consume unlimited host resources.

Exceeded quotas block **new** matching VMs until usage drops. Enable/disable lets you park a quota without deleting it.

When to use it

- To limit how many CPUs, how much memory/disk, or how many VMs a team can create
- To scope a limit to a subset of VMs by tag, rather than the whole host
- To check whether a quota is currently exceeded and which resource pushed it over
- To temporarily disable a quota without deleting its configuration
- Before handing a shared host to multiple teams — tag VMs and attach per-team quotas

How to get there

- Route / id: `/app/quotas`
- Nav: **Operations → Quotas** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Create Quota** — name; max CPUs, memory (MB), disk (GB), max VMs; optional tags to scope (empty tags = global). Checkbox to enable immediately.
2. Each quota card shows **Enabled/Disabled** and **Exceeded** badges, plus four usage bars (CPUs, memory, disk, VMs) with used/limit and color (green <75%, yellow 75–89%, red 90%+).
3. If exceeded, the card lists which resources are over limit and notes that new matching VMs cannot be created until usage drops.
4. Per-quota: **Enable/Disable**, **Edit** (same form; warns if a new limit would go below current usage), **Delete** (confirmation, cannot be undone).
5. Empty state offers a shortcut to create the first quota.

Typical flow: agree tag scheme → create tagged quotas with headroom → watch bars after create storms → disable temporarily for emergency capacity, then re-enable. Cross-check Capacity for host-level ceilings.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Profiles
- Templates
- Autoscale
- Capacity
- Virtual Machines

- Access Control
- Getting Started
- Page index

Replication

Purpose

Replication — register remote replication sites, configure per-VM replication to them with a target RPO (recovery point objective), and monitor sync health and RPO compliance across your fleet.

Use this for ongoing sync to a secondary (or bidirectional) site. For one-shot move/failover plans, see Migrations and Site Recovery.

When to use it

- Prefer this page when the job matches the purpose above
- To register a secondary (or bidirectional) site that VMs can replicate to
- To start replicating a VM to another site with a target RPO in minutes
- To pause or resume replication for a VM without tearing down the configuration
- To check overall replication health, or find which VMs are currently violating their RPO target
- Before a DR drill, to confirm sites are healthy and RPO violations are empty

How to get there

- Route / id: `/app/replication`
- Nav: **Operations → Replication** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Dashboard tab** — summary tiles for active replications, RPO violation count, average RPO, and paused/error counts, plus a per-site health list (healthy/degraded/unhealthy) with each site's replication count.
2. **Sites tab** — table of registered sites (name, type, endpoint, status, replication count, last sync). **Add Site** registers name, endpoint URL, and type (Primary, Recovery/secondary, or Bidirectional). Remove a site with confirmation.
3. **Replications tab** — per-VM configs: target RPO, status, live sync progress, last/next sync. **Configure Replication** sets VM ID, source site, target site, and RPO in minutes. **Pause** / **Resume** without tearing down config.
4. **RPO Violations tab** — replications missing target RPO (target vs current, compliance, bandwidth, sync/failure counts). Shows "All replications are compliant" when clear.

Typical flow: register Recovery site → configure replication for critical VMs with an RPO → watch Dashboard health → investigate Violations before relying on Site Recovery.

Endpoints should be reachable from this Fabric host (use `<host>` URLs you control — never paste lab-only addresses into docs).

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Site Recovery

- Migrations
- Fault Tolerance
- Backups
- Snapshots
- Getting Started
- Page index

Schedules

Purpose

VM Schedules — automate a recurring lifecycle action (start, stop, restart, or snapshot) for a single VM, on a one-time, daily, or weekly schedule.

When to use it

- To automatically stop dev/test VMs overnight or on weekends to save resources
- To take a recurring snapshot of a VM on a schedule
- To run a one-off action at a specific future time without staying logged in
- To check whether a scheduled action actually ran, and whether it succeeded

How to get there

- Route / id: `/app/schedules`
- Nav: **Operations → Schedules** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Create Schedule** — pick a VM, an action (Start VM, Stop VM, Restart VM, Create Snapshot), a schedule type, and a time (24-hour UTC):
   - **Once** — runs a single time
   - **Daily** — runs every day at the given time
   - **Weekly** — pick one or more days of the week to run on A checkbox enables the schedule immediately on creation.
2. Search schedules by name, VM, or action using the search box above the list.
3. Each schedule shows as a card with its enabled/disabled state, action badge, target VM, a human-readable description of its cadence (e.g. "Weekly on Mon, Wed at 09:00"), and next/last run times (relative).
4. Per-schedule actions: **Run Now** (executes immediately, confirmation required, disabled when the schedule itself is disabled), **Enable/Disable** (toggle without deleting), **Edit** (change name, action, cadence, or time), and **Delete** (confirmation, cannot be undone).
5. **History** button switches the page to an execution history table (latest 20 runs) showing schedule, VM, action, when it ran, and success/failure status with the error message if it failed.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Snapshots
- Backups
- Migrations
- Getting Started
- Page index

Site Recovery

Purpose

Site Recovery — define disaster recovery plans that group VMs by source and target site, then execute those plans as a test failover, a planned migration, or a full disaster recovery, and track how each execution unfolds.

When to use it

- To set up which VMs fail over together, and to which target site, ahead of an actual incident
- To run a non-destructive test failover to validate a plan works before you need it for real
- To execute an actual disaster recovery when a site goes down
- To check fleet-wide DR posture — how many VMs are protected, average RTO/RPO, and which plans have been tested

How to get there

- Route / id: `/app/site-recovery`
- Nav: **Operations → Site Recovery** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **DR Dashboard tab** — summary tiles for protected vs. unprotected VMs, average RTO, average RPO, and plans tested vs. total plans; plus a per-site status list and the 5 most recent executions with their status.
2. **Recovery Plans tab** — table of plans (name/description, VM group count and total VMs, status, last tested date, last executed date). **Create Plan** defines a plan name, description, source site ID, target site ID, a VM group name, and a comma-separated list of VM IDs in that group.
3. Each plan can be **executed** — opens a modal with three execution types, each with different risk:
   - **Test Failover** — non-destructive, runs in an isolated network
   - **Planned Migration** — graceful, zero data loss
   - **Disaster Recovery** — emergency failover; some data loss possible; confirmation dialog warns this fails over all VMs to the target site Plans can also be **deleted** (confirmation dialog).
4. **Execution History tab** — each past/in-progress execution shows plan name, execution type, status, VMs recovered vs. total, actual RTO once known, a live progress bar with the current step, a chip per step showing its individual status, and start/completion timestamps.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Snapshots
- Backups
- Migrations
- Getting Started
- Page index

Snapshots

Purpose

VM Snapshots — create point-in-time snapshots of a specific VM's disk (or disk + memory), and revert or delete them. Unlike most Operations pages, this one is scoped to one VM at a time, entered by name.

You can also manage snapshots from the **VM detail → Snapshots** tab (`/app/vms/:name`), which is the preferred path when you are already looking at a VM.

When to use it

- To capture a VM's disk state before a risky change, then roll back if needed
- To browse the snapshot history for a specific VM
- To revert a VM to an earlier snapshot
- To clean up old snapshots you no longer need

Snapshot types

Type	What it captures	Running VM	Typical duration
Disk (default)	qcow2 internal disk snapshot only	Live via QMP `blockdev-snapshot-internal-sync`	Usually under a second once the monitor is up
Full	Disk + guest memory (vmstate)	Live via QMP `snapshot-save`	Can take minutes under host disk load; allow up to ~5 minutes

Stopped VMs use `qemu-img snapshot` for both types (no live monitor required).

Tips

- Prefer **Disk** for routine checkpoints. Use **Full** only when you need to restore running process state.
- If create returns **409** (“could not reach the VM monitor yet”), wait a few seconds after start/restart and retry — the QMP socket is not ready until QEMU has fully come up. The console retries automatically a few times.
- Revert still requires the VM to be **stopped**.

Troubleshooting

Symptom	Likely cause	What to do
Snapshot create **409**	QMP monitor not ready yet after start/restart	Wait a few seconds and retry (UI auto-retries). Confirm FluxVM shows the VM `running` and `qmp.sock` exists under `/var/lib/fluxvm/instances/` .
Full snapshot hangs / times out	Memory dump under host disk load	Prefer **Disk**; allow up to ~5 minutes for Full; reduce host load (other large guests).
Auto-healer restarting right after start	Guest/QEMU flapping during boot	Fabric skips healing for 90s after start; if it continues, check FluxVM console logs and the guest image.
Start stuck / Failed with FluxVM timeout	Disk clone exceeded the old 30s client timeout	Fixed at 180s — redeploy fabricd if the host is still on an older build.

How to get there

- Route / id: `/app/snapshots`
- Nav: **Operations → Snapshots** (sidebar, command palette, or desktop nav)
- Or: **Virtual Machines →** open a VM → **Snapshots** tab

Operate from the console (UX)

Global Snapshots page (`/app/snapshots`)

1. Type a VM name into the **VM Name** field and click **Load Snapshots** (or wait — snapshots load automatically once a name is entered) to see that VM's snapshot list. Nothing loads until a VM name is provided.
2. **Create** — opens a dialog for a snapshot name, an optional description, and a type: **Disk Only** (default) or **Full (disk + memory — slower)**.
3. The snapshot table lists each snapshot's name, type, description, and relative creation time.
4. Per-snapshot actions: **Revert to snapshot** (confirmation dialog warns the VM must be stopped first) and **Delete snapshot** (confirmation dialog).
5. If the VM has no snapshots yet, the table shows "No snapshots found for this VM."

VM detail Snapshots tab

1. Open `/app/vms/:name` → **Snapshots**.
2. **Create Snapshot** — name, optional description, type (**Disk** by default).
3. **Create** runs against the live API; success refreshes the list. Delete and revert work the same as on the global page.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- Getting Started
- Snapshots & backups tutorial
- Page index

Templates

Purpose

VM Templates — reusable VM configurations (CPU/memory/disk, tags) that you save from an existing VM and use to stamp out new VMs quickly, instead of configuring resources from scratch each time.

Templates capture **resource configuration**, not a full disk clone. For disk images and ISOs, see Content Library and the images managers under More.

When to use it

- To create a new VM with the same resource configuration as one you've already tuned
- To standardize resource sizing across a team (e.g. a "small-dev" or "large-prod" template)
- To browse the templates already saved and their specs before deciding which to use
- After you have at least one well-sized VM you trust as a pattern
- When Profiles cover sizing only and you also need tags/defaults from a real VM

How to get there

- Route / id: `/app/templates`
- Nav: **Operations → Templates** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Templates aren't created on this page — click **Create from VM** to jump to Virtual Machines, where a template is saved from an existing VM's configuration.
2. Each saved template shows as a card with name, description, CPUs, memory (MB), disk size (GB), tags, and creation date.
3. **Create VM** on a template card opens a dialog asking only for a new VM name, then deploys a new VM using that template's saved resource configuration.
4. **Delete** removes a template (confirmation dialog, cannot be undone) — this does not affect VMs already created from it.
5. If no templates exist yet, the empty state points you to the VMs page to create one.

Typical flow: tune one golden VM → save template from VMs → stamp new names from the template cards → verify on Dashboard / Virtual Machines. Pair with Quotas if teams share the host.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Virtual Machines
- Create VM
- Profiles
- Content Library
- Quotas

- Backups
- Getting Started
- Page index

Access Control

Purpose

Access Control — manage the user accounts that can sign in to Zyvor Fabric: create accounts, assign a role (admin, operator, or viewer), and enable/disable or delete them.

Roles: **Admin**, **Operator**, **Viewer**. Disable locks a user out without deleting history.

When to use it

- To create a login for a new team member and decide up front what they're allowed to do
- To temporarily lock a user out without deleting their account
- To see at a glance who has admin access, or when someone last logged in
- To remove an account that's no longer needed
- Prefer this page when the job matches the purpose above
- After failed-login spikes on Security Dashboard, disable the targeted account here

How to get there

- Route / id: `/app/access-control`
- Nav: **Security → Access Control** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review tiles — Total Users, Admins, Operators, Viewers.
2. **Add User** — username (3–32 chars: letters, numbers, hyphens, underscores), password (8+), role via **Admin** / **Operator** / **Viewer**. Client-side validation before submit.
3. User table: avatar/username, role badge, **Active/Disabled** toggle, created date, last login (or "Never").
4. Flip **Active/Disabled** to lock or restore without deleting.
5. Trash icon deletes after confirmation (`Delete user "…"?`).

Typical flow: Add User as Viewer first → raise to Operator when needed → keep Admin count small → Disable instead of Delete when investigating. Confirm actions in Audit.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Security Dashboard
- Audit
- Compliance
- Certificates
- Sign in
- Getting Started
- Page index

Certificates

Purpose

Certificates & Security — a PKI console for Zyvor Fabric: certificate authorities, issued certificates, the CSR approval queue, host TPM/boot attestation, and VM security baselines, rolled up into one health dashboard.

Six tabs: Dashboard, CAs, Certificates, Requests, Attestation, Security Baselines.

When to use it

- To see which certificates are expiring soon and renew them before they lapse
- To stand up a new certificate authority (root, intermediate, or external) for issuing certs
- To approve certificate signing requests waiting in the queue
- To check whether hosts have a TPM present and are passing boot/secure-boot attestation
- To track fleet compliance against a VM security baseline
- Prefer this page when the job matches the purpose above

How to get there

- Route / id: `/app/certificates`
- Nav: **Security → Certificates** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Dashboard** — totals for certs (active/expiring/expired), CAs, pending requests, compliance %, plus certs expiring within 30 days.
2. **CAs** — table + **Create CA** (name, Root/Intermediate/External, subject e.g. `CN=My CA, O=My Org`).
3. **Certificates** — issued certs; **Revoke** on active ones (confirmation).
4. **Requests** — CSR queue; approve pending with one click (no reject action in this view).
5. **Attestation** — per-host TPM present/version, attestation status, boot integrity, secure boot, last check.
6. **Security Baselines** — VM count, compliant count, checks, % bar; **Create Baseline** (name, description; default check attached).

Typical flow: Dashboard for expiring-soon → approve Requests → Create CA only when standing up PKI → check Attestation after host changes. Cross-check overall posture on Compliance / Security Dashboard.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Security Dashboard
- Compliance
- Encryption
- Access Control
- Audit

Compliance

Purpose

Compliance Dashboard — a security and configuration compliance scorecard: an overall score, pass/warning/fail counts by category, and remediation guidance for anything that isn't passing.

Point-in-time configuration checks. For live threats see Security Dashboard; for patch baselines see Lifecycle.

When to use it

- To get a single number for how compliant your deployment currently is
- To find exactly which checks are failing or warning, and how to fix each one
- To trigger a fresh compliance scan on demand instead of waiting for the next one
- To narrow the check list down to one category, e.g. only network or only auth checks
- Prefer this page when the job matches the purpose above
- Before an audit, export mental notes from Fail/Fix lines and remediate

How to get there

- Route / id: `/app/compliance`
- Nav: **Security → Compliance** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. View the compliance score (0–100, green/amber/red) plus Passed, Warnings, Failed, and Total Checks tiles.
2. Click **Run Scan** — button reads "Scanning..." then refreshes results when complete.
3. Filter checks by category pills (**all**, plus each category from the last scan).
4. Review each check's name, status (pass/warning/fail), description; non-passing rows show a "Fix:" remediation note.
5. "Last scan" timestamp at the bottom shows when data was collected.

Typical flow: Run Scan → filter to Failed → apply Fix guidance on the owning page → Run Scan again until score recovers. Pair with Certificates/Access Control for PKI and account hygiene.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Security Dashboard
- Certificates
- Access Control
- Encryption
- Lifecycle
- Audit
- Getting Started

Encryption

Purpose

Encryption — manage VM disk and vMotion encryption: register key management providers (KMIP, HashiCorp Vault Transit, or local software keys), define reusable encryption policies, and see which VMs are encrypted under which policy.

Providers → policies → encrypted VMs. Rotate keys on demand after suspected compromise.

When to use it

- To connect an external KMS so Zyvor Fabric manages encryption keys outside the host
- To define a policy (algorithm, whether vMotion traffic is encrypted, key rotation schedule) and see who's using it
- To check which VMs are currently encrypted and under which policy
- To rotate a VM's encryption key on demand, e.g. after a suspected key compromise
- Prefer this page when the job matches the purpose above
- Before enabling encryption broadly, start with a Local provider in a non-prod window

How to get there

- Route / id: `/app/encryption`
- Nav: **Security → Encryption** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Summary tiles (Key Providers, Policies, Encrypted VMs, Connected Providers), then three tabs:

1. **Key Providers** — name, type, endpoint, status. **Add Provider**: name, type (**KMIP**, **Local**, or **HashiCorp Vault Transit**), endpoint URL. Remove with trash + confirmation.
2. **Policies** — provider, algorithm, vMotion encryption, auto-rotate interval. **Create Policy**: name, description, provider dropdown, algorithm (**AES-256-XTS**, **AES-256-CBC**, or **ChaCha20-Poly1305**), Encrypt vMotion toggle, Auto-rotate toggle (+ days).
3. **Encrypted VMs** — encrypted yes/no, policy, algorithm, last rotation. **Rotate Key** on encrypted VMs (confirmation).

Typical flow: Add Provider → Create Policy → confirm VMs appear under Encrypted VMs → Rotate Key only when required. Keep endpoints as `https://<host>/…` (or your KMS URL) — not lab-specific IPs in shared docs.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Certificates
- Compliance
- Security Dashboard
- Access Control

- Migrations
- Virtual Machines
- Getting Started
- Page index

Plugins

Purpose

Plugin Manager — enable, disable, and review the server extensions installed on Zyvor Fabric (storage, network, security, monitoring, and backup plugin types).

Enable/disable only — there is no install or configure-plugin flow on this page.

When to use it

- Prefer this page when the job matches the purpose above
- To see which plugins are installed and whether each is running, stopped, or erroring
- To turn a plugin on or off without restarting the whole service
- To check a plugin's version and author before relying on it
- Prefer this page when the job matches the purpose above
- After an upgrade, to confirm expected plugins are Running and Errors is zero

How to get there

- Route / id: `/plugins`
- Nav: **Security → Plugins** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review the four stat tiles: Total Plugins, Running, Errors, and Types (distinct categories).
2. Browse plugin cards — name, version, type badge (storage/network/security/monitoring/backup), status (running/stopped/error), description, author when provided.
3. Click **Enable**/**Disable** on a card; the button spins while the request is in flight and the card status updates after.

Typical flow: scan Errors tile → open the erroring card → Disable if it is blocking, or fix backend deps then Enable again. Confirm Dashboard capability chips still look healthy after toggles.

Operator tip: treat Errors > 0 as blocking for change windows until the plugin is fixed or intentionally Disabled.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Security Dashboard
- Access Control
- Certificates
- Encryption
- Backups
- Dashboard
- Getting Started

Security Dashboard

Purpose

Security — a real-time security posture and threat-monitoring view: an overall risk score, active security alerts, recent failed login attempts, and listening network ports on the host. Data auto-refreshes every 5 seconds.

Distinct from Compliance (configuration checks) and Certificates (PKI health) — this page is about **live threats and activity**, not point-in-time audits.

When to use it

- To get a fast read on overall risk from the single risk-score gauge
- To see currently active security alerts by severity (critical/warning/info)
- To spot a brute-force or credential-stuffing attempt via the failed-logins table
- To audit which ports are open and listening on the host, and which process owns each one
- Prefer this page when the job matches the purpose above
- During on-call when you need a security pulse before opening Audit/Access Control

How to get there

- Route / id: `/security-dashboard`
- Nav: **Security → Security Dashboard** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. View the risk score gauge (0–100, color-coded) alongside Critical Alerts, Warnings, and Failed Logins count tiles.
2. Review the Security Alerts list — severity badge, message, source, timestamp; "No active alerts" when clear.
3. Review the Failed Logins table (time, user, source) — only when there are failed attempts.
4. Review the Listening Ports table (port, protocol, process, PID) — only when ports are open.
5. Auto-refresh every 5 seconds; manual refresh in the header.

Typical flow: check risk score → read critical alerts → if failed logins spike, disable the user on Access Control and pull Audit. Unexpected listeners → correlate PID on Processes.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Access Control
- Compliance
- Certificates
- Encryption
- Audit
- Alerts

Cost Estimator

Purpose

Storage Cost Estimator — a what-if calculator that projects cloud storage cost (AWS S3, Azure Blob, GCS) for your VM fleet and compares it against an on-premises baseline. It's a planning tool: figures come from the inputs you set, not from your actual billing or usage.

When to use it

- To budget or justify a move to cloud storage before you commit to it
- To see how fleet size, average VM disk size, or snapshot retention changes projected storage spend
- To put a number on "how much would we save vs. on-prem" for a proposal or ticket

How to get there

- Route / id: `/cost-estimator`
- Nav: **Tools → Cost Estimator** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Configuration** — set Number of VMs (1–1000), Average VM Size (10 GB–2 TB), and Storage Duration (1–36 months) with sliders. Toggle **Include snapshots** to add 50% to the total storage figure.
2. Click **Calculate Costs**. The page first tries the backend (`POST /api/cost/estimate`); if that call fails or returns no estimates, it falls back to a client-side calculation using fixed per-GB rates (AWS S3 $0.023, Azure Blob $0.018, GCS $0.020, on-prem $0.10).
3. Results show three provider cards (Monthly / Annual / Total for the chosen duration), with the cheapest option badged.
4. A savings panel compares the cheapest cloud option against the on-prem estimate, with a percentage saved.
5. A horizontal bar chart compares monthly cost across all three cloud providers plus on-prem.
6. **Copy Estimate to Clipboard** copies a plain-text summary of your inputs and the results — handy for pasting into a ticket or email.
7. **Empty / fail:** Check health, auth, and domain dependencies.
8. **Success:** Live data loads; mutations complete without error toasts.

Related pages

- API Playground
- Webhooks
- VM Compare
- Getting Started
- Page index

Notification Center

Purpose

Notification Center — a live, session-only tray of VM events and system alerts, polled from the server every 10 seconds. It's separate from **Monitoring → Notifications**, which manages persistent delivery channels, rules, and history; this page just surfaces what's firing right now and forgets everything when you reload.

When to use it

- To catch alerts and warnings as they happen without watching the Monitoring dashboard
- To scan a short-lived list of recent events during troubleshooting
- Not for setting up where alerts get delivered (email/Slack/webhook) — use Notifications for that

How to get there

- Route / id: `/notification-center`
- Nav: **Tools → Notification Center** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. The page polls `GET /api/system/alerts` every 10 seconds; each newly seen alert is added to the feed, tagged **Alert** or **System Warning** based on its severity. It also subscribes to the live `/api/events/stream` feed for **VM Started** / **VM Stopped** events, so all four categories populate from real activity.
2. Category chips at the top show a live count per type.
3. An unread count appears when there are unread notifications; clicking a notification marks it read (dims it).
4. Click the **×** on a notification to dismiss it individually, or **Clear All** to empty the whole feed.
5. If polling fails, an amber banner reports the error; polling keeps retrying every 10 seconds in the background.
6. **Empty / fail:** Check health, auth, and domain dependencies.
7. **Success:** Live data loads; mutations complete without error toasts.

Related pages

- API Playground
- Webhooks
- VM Compare
- Getting Started
- Page index

API Playground

Purpose

API Playground — send authenticated, ad-hoc HTTP requests to the Zyvor Fabric API from the browser: pick a preset endpoint or type any path, choose method, optional JSON body, inspect status/headers/body, and keep a short in-session history.

When to use it

- To probe `/health`, `/api/vms`, storage, templates, backups, audit, and other presets without leaving the console
- To try a POST/PUT body against an endpoint before scripting it
- To copy a formatted JSON response for tickets or docs
- To confirm auth is working (requests use your current session)

How to get there

- Route / id: `/app/playground`
- Nav: top-bar **API Playground** icon, **Tools** (when listed), or command palette
- Legacy `/playground` redirects here

Operate from the console (UX)

1. Open the page — left column lists **preset endpoints** (Health, List VMs, CPU/NUMA topology, network links, storage pools, templates, backups, audit, certificates, quotas, schedules, webhooks, optimizations).
2. Click a preset to fill method + path, or type your own path (e.g. `/api/vms/myvm`) and pick **GET** / **POST** / **PUT** / **DELETE**.
3. For POST/PUT, paste a JSON body in the request editor when needed.
4. **Send** — response panel shows status, duration, headers, and pretty-printed JSON when possible. Use **Copy** on the body.
5. **History** (session-only, last ~20) lets you re-select a prior call; clear with the trash control.
6. **Empty / fail:** Network/auth errors show in the error banner with daemon hints; HTTP 4xx/5xx still populate the response panel so you can read the API error body.
7. **Success:** Green/2xx status chip, formatted body, and a new history row.

For contracts and examples prefer operator docs under `docs/` (e.g. API reference) — the playground does not invent endpoints.

Related pages

- Dashboard
- Virtual Machines
- Audit
- Getting Started
- Page index

VM Compare

Purpose

VM Comparison — a side-by-side diff of two VMs' configurations, run on demand against the live VM list.

Compares live config fields via `GET /api/vms/compare?source=…&target=…`. It does not mutate either VM.

When to use it

- To see what differs between two VMs before assuming they're equivalent (useful when troubleshooting "why does this one behave differently")
- To confirm a newly cloned or templated VM actually matches its source
- To audit configuration drift between two VMs that are supposed to be identical
- Prefer this page when the job matches the purpose above
- After Templates create, to verify the stamp matches the golden VM

How to get there

- Route / id: `/vm-compare`
- Nav: **Tools → VM Compare** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Pick **Source VM** and **Target VM** from dropdowns populated from the VM list (target excludes the chosen source).
2. Click **Compare** (disabled until both are selected).
3. Results table: one row per field — Source value, Target value, Match (Yes green / No amber).
4. Header refresh re-fetches the VM list if it is stale.
5. Re-run Compare after you change either VM's config to confirm drift is gone.

Typical flow: source = golden / template parent → target = new VM → note No rows → fix on Virtual Machines → Compare again. For health rather than config, use VM Health Check.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- VM Health Check
- Virtual Machines
- Templates
- Profiles
- API Playground
- Webhooks
- Getting Started
- Page index

VM Health Check

Purpose

VM Health Check — runs a set of health verification checks against a single VM, on demand, and reports pass/warning/fail per check plus an overall status.

On-demand via `GET /api/vms/{name}/healthcheck`. A failed **request** (banner + retry) is different from a check that returns fail/warning.

When to use it

- Before or after a change, to confirm a VM is still in good health
- To triage a VM you suspect is unhealthy, with a per-check breakdown instead of just up/down
- Before promoting a VM to production or handing it off
- Prefer this page when the job matches the purpose above
- After start/migrate/restore, before pointing traffic at the guest

How to get there

- Route / id: `/vm-healthcheck`
- Nav: **Tools → VM Health Check** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Select a VM from the dropdown (live VM list).
2. Click **Run Health Check**.
3. Overall banner: **Healthy** or **Issues Found**, with "X of Y checks passed".
4. Each check: status icon (pass / warning / fail), message, optional detail line.
5. Request failure → error banner with retry (not the same as a failing check).
6. Header refresh reloads the VM list.

Typical flow: pick VM → Run → if Issues Found, open detail/console and fix → Run again until Healthy. Pair with VM Compare when the symptom is config drift rather than runtime health.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- VM Compare
- Virtual Machines
- VM Console
- Live Metrics
- Alerts
- API Playground
- Getting Started

Webhooks

Purpose

Webhook Configuration — manage outbound webhooks that notify an external endpoint (generic HTTP, Slack, or Discord) when specific VM and backup events occur.

Created webhooks are **enabled by default**. Use **Test** before relying on them in production. Distinct from Notifications channel/rules UI — this page is the Tools webhook registry for VM/backup events.

When to use it

- To wire VM lifecycle events (started, stopped, created, deleted) or backup results (completed, failed) into Slack, Discord, or your own HTTP endpoint
- To verify a webhook endpoint is reachable and correctly configured before relying on it
- To audit which webhooks exist, what events they listen for, and whether they're enabled
- Prefer this page when the job matches the purpose above
- When an external automation needs a simple HTTP callback instead of polling Fabric APIs

How to get there

- Route / id: `/webhooks`
- Nav: **Tools → Webhooks** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Add Webhook** — destination URL, delivery type (generic / Slack / Discord), multi-select events (`vm.started` , `vm.stopped` , `vm.created` , `vm.deleted` , `backup.completed` , `backup.failed`). URL + ≥1 event required.
2. Save posts `POST /api/webhooks` (enabled by default) and refreshes the list.
3. Each row: URL (copy button), type badge, event tags, enabled/disabled indicator.
4. **Test** — `POST /api/webhooks/test` ; reports delivered/failed inline.
5. Trash → confirm → `DELETE /api/webhooks/{id}` .
6. Header refresh reloads the list.

Typical flow: Add with a test URL on `https://<host>/…` or your chat webhook → Test → subscribe only the events you need → remove unused hooks. For richer channel/rules/history, also configure Monitoring → Notifications.

Empty / fail: Error banner, empty table, or failed toast — confirm you are signed in, `zyvor-fabricd` is healthy (`/readyz` on `http://127.0.0.1:<port>` or `https://<host>`), and any backend this page needs (FluxVM, storage, network) is reachable. See Admin basics.

Success: Live data loads without error; creates/updates appear in the list or detail view and any confirmation toast clears cleanly.

Related pages

- Notifications
- Notification Center
- Backups
- Event Stream

- API Playground
- VM Health Check
- Getting Started
- Page index

Zyvor Fabric — Complete page index

Marketing: `/`, `/product`, `/platform`, `/security`, `/sign-in`.

Console routes under `/app` — every primary navigable ops route.

Generated: 2026-09-10 · 84 routes

Regenerate: `node scripts/user-docs/generate-page-index.mjs`

Marketing & auth

Page	Route	Purpose
Home	`/`	Public marketing home
Product	`/product`	Product story
Platform	`/platform`	Interfaces (Web, CLI, Operator, Terraform)
Security	`/security`	Security story
Sign in	`/sign-in`	Console authentication (`/login` redirects here)

Core

Page	Route	Purpose	Guide
Dashboard	`/app`	Dashboard — Core surface.	Open
Favorites	`/app/favorites`	Favorites — Core surface.	Open
Virtual Machines	`/app/vms`	Virtual Machines — Core surface.	Open
Machines	`/app/machines`	Machines — Core surface.	Open
Profiles	`/app/profiles`	Profiles — Core surface.	Open
Datacenters	`/app/datacenters`	Datacenters — Core surface.	Open
VM Browser	`/app/vm-browser`	VM Browser — Core surface.	Open
Create VM	`/app/create`	Create VM — Core surface.	Open
VM Console	`/app/vms/:name/console`	VM Console — Core surface.	—
Settings	`/app/settings`	Settings — Core product and console preferences.	Open
VM Wizard	`/app/vm-wizard`	VM Wizard — guided VM creation flow.	Open

Infrastructure

Page	Route	Purpose	Guide
Network	`/app/network`	Network — Infrastructure surface.	Open
Net Security	`/app/network-security`	Net Security — Infrastructure surface.	Open
Storage	`/app/storage`	Storage — Infrastructure surface.	Open
Storage Pools	`/app/storage-pools`	Storage Pools — Infrastructure surface.	Open

Page	Route	Purpose	Guide
Distributed Storage	`/app/distributed-storage`	Distributed Storage — Infrastructure surface.	Open
Resource Pools	`/app/resource-pools`	Resource Pools — Infrastructure surface.	Open
System	`/app/system`	System — Infrastructure surface.	Open
System Health	`/app/system-health`	System Health — Infrastructure surface.	Open
Containers	`/app/containers`	Containers — Infrastructure surface.	Open
Zones	`/app/zones`	Zones — Infrastructure surface for placement domains.	Open
VM Pools	`/app/vm-pools`	VM Pools — warm / capacity pools for fast provisioning.	Open

Operations

Page	Route	Purpose	Guide
DRS	`/app/drs`	DRS — Operations surface.	Open
Fault Tolerance	`/app/fault-tolerance`	Fault Tolerance — Operations surface.	Open
Replication	`/app/replication`	Replication — Operations surface.	Open
Site Recovery	`/app/site-recovery`	Site Recovery — Operations surface.	Open
Migrations	`/app/migrations`	Migrations — Operations surface.	Open
Migration Wizard	`/app/migration-wizard`	Migration Wizard — Operations surface.	Open
Templates	`/app/templates`	Templates — Operations surface.	Open
Content Library	`/app/content-library`	Content Library — Operations surface.	Open
Schedules	`/app/schedules`	Schedules — Operations surface.	Open
Autoscale	`/app/autoscale`	Autoscale — Operations surface.	Open
Snapshots	`/app/snapshots`	Snapshots — Operations surface.	Open
Backups	`/app/backups`	Backups — Operations surface.	Open
Quotas	`/app/quotas`	Quotas — Operations surface.	Open
Lifecycle	`/app/lifecycle`	Lifecycle — Operations surface.	Open
Bulk Operations	`/app/bulk-operations`	Bulk Operations — Operations surface.	Open

Monitoring

Page	Route	Purpose	Guide
Logs	`/app/logs`	Logs — Monitoring surface.	Open
Analytics	`/app/analytics`	Analytics — Monitoring surface.	Open
Audit	`/app/audit`	Audit — Monitoring surface.	Open
Notifications	`/app/notifications`	Notifications — Monitoring surface.	Open
Alerts	`/app/alerts`	Alerts — Monitoring surface.	Open

Page	Route	Purpose	Guide
Timeline	`/app/timeline`	Timeline — Monitoring surface.	Open
Processes	`/app/processes`	Processes — Monitoring surface.	Open
Kernel	`/app/kernel`	Kernel — Monitoring surface.	Open
Debug Tools	`/app/debug`	Debug Tools — Monitoring surface.	Open
Explain	`/app/explain`	Explain — Monitoring surface.	Open
Live Metrics	`/app/live-metrics`	Live Metrics — Monitoring surface.	Open
Event Stream	`/app/event-stream`	Event Stream — Monitoring surface.	Open
Optimizer	`/app/resource-optimizer`	Optimizer — Monitoring surface.	Open
Capacity	`/app/capacity-planning`	Capacity — Monitoring surface.	Open
Service Map	`/app/service-map`	Service Map — Monitoring surface.	Open

Security

Page	Route	Purpose	Guide
Security Dashboard	`/app/security-dashboard`	Security Dashboard — Security surface.	Open
Encryption	`/app/encryption`	Encryption — Security surface.	Open
Certificates	`/app/certificates`	Certificates — Security surface.	Open
Compliance	`/app/compliance`	Compliance — Security surface.	Open
Access Control	`/app/access-control`	Access Control — Security surface.	Open
Plugins	`/app/plugins`	Plugins — Security surface.	Open

Tools

Page	Route	Purpose	Guide
Webhooks	`/app/webhooks`	Webhooks — Tools surface.	Open
Cost Estimator	`/app/cost-estimator`	Cost Estimator — Tools surface.	Open
VM Compare	`/app/vm-compare`	VM Compare — Tools surface.	Open
VM Health Check	`/app/vm-healthcheck`	VM Health Check — Tools surface.	Open
Notification Center	`/app/notification-center`	Notification Center — Tools surface.	Open
API Playground	`/app/playground`	API Playground — try Fabric APIs interactively.	Open

More — images, migrations & managers

Page	Route	Purpose	Guide
Readiness	`/app/migration-readiness`	Readiness — More — images, migrations & managers surface.	Open

Page	Route	Purpose	Guide
History	`/app/migration-history`	History — More — images, migrations & managers surface.	Open
Report	`/app/migration-report`	Report — More — images, migrations & managers surface.	Open
Migration Templates	`/app/migration-templates`	Migration Templates — More — images, migrations & managers surface.	Open
Batch Migration	`/app/batch-migration`	Batch Migration — More — images, migrations & managers surface.	Open
ISO Images	`/app/iso-images`	ISO Images — More — images, migrations & managers surface.	Open
Upload Disk	`/app/upload-disk`	Upload Disk — More — images, migrations & managers surface.	Open
Download Disk	`/app/download-disk`	Download Disk — More — images, migrations & managers surface.	Open
Image Builder	`/app/image-builder`	Image Builder — More — images, migrations & managers surface.	Open
Pipeline	`/app/pipeline`	Pipeline — More — images, migrations & managers surface.	Open
Disk Images	`/app/disk-images`	Disk Images — More — images, migrations & managers surface.	Open
Disk Converter	`/app/disk-converter`	Disk Converter — More — images, migrations & managers surface.	Open
Backup Scheduler	`/app/backup-scheduler`	Backup Scheduler — More — images, migrations & managers surface.	Open
Batch Import	`/app/batch-import`	Batch Import — More — images, migrations & managers surface.	Open
Snapshot Mgr	`/app/snapshot-manager`	Snapshot Mgr — More — images, migrations & managers surface.	Open
Storage Mgr	`/app/storage-manager`	Storage Mgr — More — images, migrations & managers surface.	Open
Manifest Builder	`/app/manifest-builder`	Manifest Builder — More — images, migrations & managers surface.	Open
Job Monitor	`/app/job-monitor`	Job Monitor — More — images, migrations & managers surface.	Open
Network Topology	`/app/network-topology`	Network Topology — More — images, migrations & managers surface.	Open

Auth

Page	Route	Purpose	Guide
Login	`/app/sign-in`	Login — Auth surface.	Open

Related

- User docs home
- Page-by-page guides